

Document Title

**Orbital Compute Data Center Digital Twin
Platform
Overall Design Report**

Prepared by

Nanyang Technological University

Author

Checked by

Reviewed by

Approved by

August 2026

Abstract:

This document is the overall design report of the orbital compute data center digital twin platform. It gives the technical indicators, the requirement analysis, the overall architecture, the system composition, the preliminary and detailed design of each subsystem, and the requirement compliance status.

Keywords: digital twin; orbital computing; orbital dynamics; power and thermal control; GPU inference performance; real-time simulation

Revision record:

Change number	Date	Changed by	Method

Contents

1 Document Overview.....	5
2 Project Origin	5
3 Technical Indicators	5
4 Requirement Analysis.....	6
5 Overall Design	7
5.1 Overall Architecture	7
5.2 System Composition.....	9
5.3 Preliminary Design of the Subsystems.....	10
5.3.1 Simulation Model Subsystem	10
5.3.2 Simulation Engine Subsystem.....	12
5.3.3 Data Exchange Subsystem.....	12
5.3.4 Situation Display and Simulation Management Subsystem	13
5.3.5 3D Rendering Subsystem	14
5.3.6 3D Asset Subsystem.....	15
5.4 Usage Flow	15
6 Detailed Design	17
6.1 Simulation Model Subsystem.....	17
6.1.1 Model Hierarchy.....	17
6.1.2 Concept Models	17
6.1.3 Algorithm Models	18
6.1.4 Parameter Catalogs	27
6.1.5 Entity Models	29
6.1.6 Thermal-Compute Feedback Loop	32
6.2 Simulation Engine Subsystem	32
6.2.1 Engine Structure and Runtime Conventions	32
6.2.2 Model Initialization Module.....	33
6.2.3 Real-Time Stepping Module	33
6.2.4 Command Response Module	35
6.2.5 State Output Module	36
6.2.6 Comparison Simulation Module	36
6.2.7 Time System.....	36
6.3 Data Exchange Subsystem	37
6.3.1 Unified Envelope.....	37
6.3.2 WebSocket Broadcast Module	37
6.3.3 REST Interface Module	38
6.3.4 Renderer Polling Interface	39

6.3.5 Data Entity Contract	39
6.4 Situation Display and Simulation Management Subsystem.....	40
6.4.1 Overall Page Structure	40
6.4.2 Overview Page.....	41
6.4.3 Orbit and Ground Station Workbench	41
6.4.4 Satellite Twin Page.....	41
6.4.5 Satellite Builder Wizard.....	43
6.4.6 Design Library and Comparison Panel	43
6.4.7 Scenario Data Structures	43
6.5 3D Rendering Subsystem.....	44
6.5.1 Host and Extensions	44
6.5.2 Stages and Units	45
6.5.3 Per-Frame Drive and Time Anchoring	45
6.5.4 Streaming and the Three Viewport States.....	46
6.6 3D Asset Subsystem	46
6.6.1 Parametric Model Generation.....	46
6.6.2 Geometry Parameters and the Hot Reload Pipeline.....	47
6.6.3 Offline Software Renderer and Asset Rules.....	47
6.7 Verification Design	47
6.7.1 Validators	47
6.7.2 Benchmark Against STK 11.6	48
6.7.3 Whole-Satellite Thermal Comparison Against COMSOL 6.3.....	49
6.7.4 Interface End-to-End Tests	51
7 Requirement Compliance Status	51

1 Document Overview

This document is the overall design report of the orbital compute data center digital twin platform. The software name of the platform is SDCTwin, referred to below as the platform. The document gives the technical indicators, the requirement analysis, the overall architecture, the system composition, the preliminary and detailed design of each subsystem, and the requirement compliance status. It is intended for the developers, testers and integrators of the platform. Developers can carry out development and iteration according to this document, testers can derive the verification plan from Chapter 3 and Section 6.7, and integrators can connect external systems through the interfaces defined in Section 6.3.

Referenced documents:

Document 1 Design Report for the Orbit Dynamics Module of SDCTwin, College of Computing and Data Science, Nanyang Technological University, August 2026.

Document 2 Test Report for the Orbit Dynamics Module of SDCTwin, College of Computing and Data Science, Nanyang Technological University, August 2026.

Document 3 V100 power-cap sweep measurement study, the calibration source of the GPU inference performance model.

Document 4 Vallado D A, Crawford P, Hujsak R, Kelso T S. Revisiting Spacetrack Report #3. AIAA 2006-6753.

Document 5 AGI STK 11.6 user documentation and the Space Environment and Effects Tool documentation.

2 Project Origin

The project originates from the development task of the orbital compute data center digital twin platform. The platform addresses compute satellites in the tens of kilowatts class that have not yet been built. It provides simulation, evaluation and presentation for the joint design of their orbit, power, thermal control and compute payload, and it provides a common simulation environment for subsequent constellation-level mission planning and effectiveness evaluation.

3 Technical Indicators

Indicator 1 The platform supports single-satellite digital twins and constellation situation simulation with no fewer than 1500 satellites. The satellite entity supports SGP4 orbit prediction. Over 24 hours the propagated position differs from the STK 11.6 reference by no more than 1 m, the ground track latitude and longitude by no more than 0.02° , and the altitude by no more than 1 km.

- Indicator 2** The Sun direction angle and the orbit β angle differ from the reference by no more than 0.5° , eclipse event times by no more than 15 s, and the 24 hour sunlit fraction by no more than 1 percentage point.
- Indicator 3** Ground station access window start and end times differ from the reference by no more than 1 s with no missed or spurious windows at the 0° , 5° , 10° and 20° elevation masks.
- Indicator 4** The 24 hour solar array energy under Sun-pointing and nadir-pointing attitudes differs from the reference by no more than 2%, and the minimum battery state of charge by no more than 5 percentage points.
- Indicator 5** The mean equilibrium temperature in the sunlit and eclipse segments differs from the reference by no more than 10 K.
- Indicator 6** The simulation engine steps in real time at 1 Hz, state data is broadcast to the browser at 1 Hz, and the 3D viewport is streamed at 1280×720 and 30 frames per second.
- Indicator 7** Satellite design parameters are configurable with no fewer than 4 satellite platforms, 4 GPU models, 4 radiator coatings, 3 solar cell materials, 4 battery chemistries and 6 hull architectures, and 2 to 4 design variants can be compared in lockstep with the live simulation.
- Indicator 8** The GPU inference performance model is calibrated against measured data with no fewer than 50 theory checks and no fewer than 30 thermal-compute closed-loop checks.
- Indicator 9** REST and WebSocket network interfaces are provided so that external systems can read state data and inject commands.

4 Requirement Analysis

The nine technical indicators of Chapter 3 are analysed as software requirements and decomposed to the subsystems to obtain the complete function set, as shown in Table 1.

Table 1 Requirement decomposition

No.	Indicator	Subsystem	Functional requirement	Performance requirement
1	Constellation scale and orbit prediction accuracy	Simulation model subsystem	Satellite entity supports SGP4 prediction, ground track conversion and Walker constellation generation	24 h position difference ≤ 1 m, ground track difference $\leq 0.02^\circ$, constellation ≥ 1500 satellites
		Simulation engine subsystem	Every physics step propagates the whole constellation and computes aggregate metrics	1500 satellites propagated within the 1 s step
2	Sun and eclipse accuracy	Simulation model subsystem	Analytic Sun position, conical umbra and penumbra eclipse model	Sun direction $\leq 0.5^\circ$, eclipse timing ≤ 15 s, sunlit fraction ≤ 1 percentage point
3	Access window accuracy	Simulation model subsystem	Ground station elevation and access window model on the WGS-84	Window edges ≤ 1 s, no missed or spurious windows

No.	Indicator	Subsystem	Functional requirement	Performance requirement
			ellipsoid	
4	Power and storage accuracy	Simulation model subsystem	Solar array power and battery state of charge models; power varies with attitude, Sun distance, eclipse and temperature	24 h energy $\leq 2\%$, minimum state of charge ≤ 5 percentage points
5	Thermal balance accuracy	Simulation model subsystem	Lumped thermal balance with solar absorption, Earth albedo and Earth infrared	Segment mean temperature ≤ 10 K
6	Real-time behaviour	Simulation engine subsystem	1 Hz stepping coroutine, state snapshot and read-time refresh	Step period 1 s
		Data exchange subsystem	WebSocket broadcast and renderer polling	Broadcast 1 Hz, polling 5 Hz
		3D rendering subsystem	RTX rendering and WebRTC streaming	1280×720 at 30 frames per second
7	Design configurability	Situation display and simulation management subsystem	Builder wizard, configurator, design library, comparison panel	4 platforms, 4 GPUs, 4 coatings, 3 cell materials, 4 chemistries, 6 architectures, 2 to 4 variants
		3D asset subsystem	Parametric hull model generation and hot reload	Renderer reloads within the next polling period
8	Inference model credibility	Simulation model subsystem	LLM inference performance model with DVFS power law, memory-bound decode law and thermal throttling coupling	≥ 50 theory checks, ≥ 30 closed-loop checks
9	Network interface adaptability	Data exchange subsystem	REST and WebSocket interfaces with a unified message envelope	External systems can read state and inject commands

5 Overall Design

The platform is a digital twin software system for orbital compute data centers. It is built on a simulation engine that steps in real time at 1 Hz and solves the orbit, lighting, power generation, energy storage, thermal balance and GPU inference performance of a satellite as one coupled system, and it presents the situation through a browser interface and a 3D rendered viewport. The platform serves the design evaluation of compute satellites, runtime situation rehearsal, design alternative comparison and algorithm validation.

5.1 Overall Architecture

The overall architecture of the platform is shown in Figure 1. The platform is divided into the computation layer, the service layer and the application layer, and a 3D asset library supplies 3D content to the service layer as an independent asset domain.

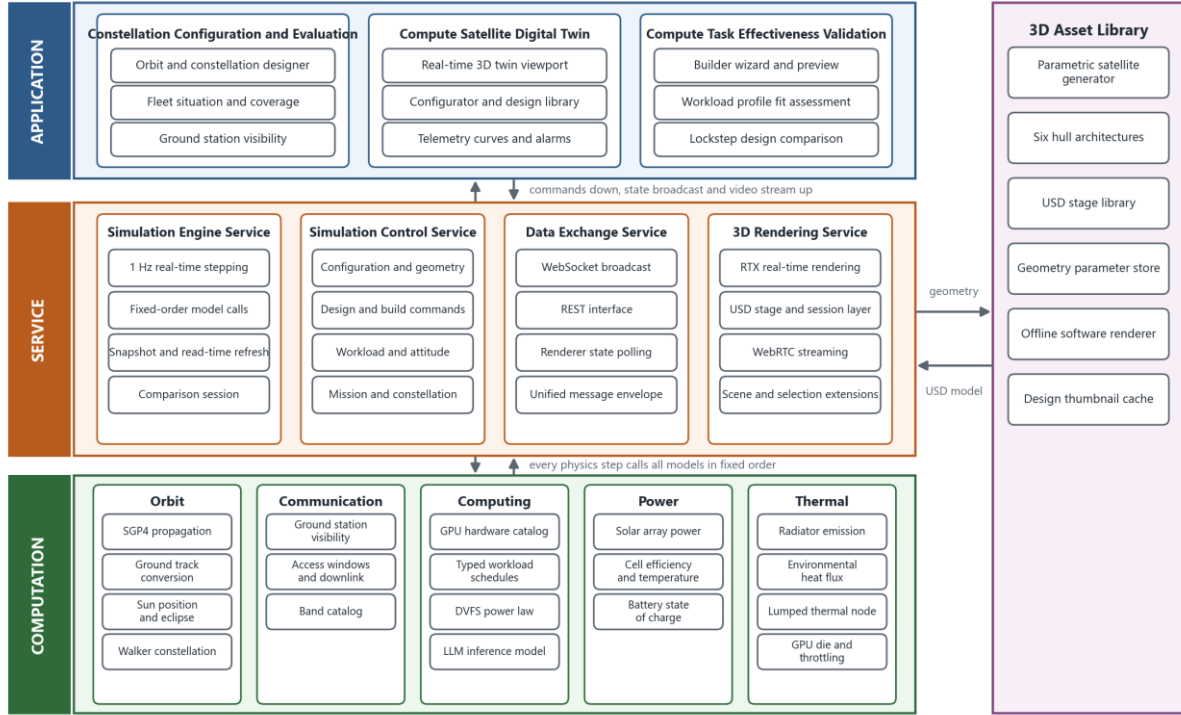


Figure 1 Overall architecture of the orbital compute data center digital twin platform

The computation layer provides the basic scientific computation and domain models required by the simulation, organized in five domains: orbit, communication, computing, power and thermal. The orbit domain contains SGP4 orbit propagation, ground track and coordinate conversion, Sun position and eclipse determination, and Walker constellation generation. The communication domain contains ground station visibility, access windows and downlink, and the communication band catalog. The computing domain contains the GPU hardware catalog, typed workload schedules, the DVFS power-frequency law and the LLM inference performance model. The power domain contains solar array power, cell efficiency with its temperature coefficient, and battery state of charge integration. The thermal domain contains radiator emission, environmental heat flux, the lumped thermal node, and GPU die temperature with throttling. All computation layer code lives in the backend directory, one file per model, and each model can be tested and replaced independently.

The service layer integrates the runtime mechanisms above the computation layer and is divided into the simulation engine service, the simulation control service, the data exchange service and the 3D rendering service. The simulation engine service is the single authoritative source of business state. Every physics step calls all computation layer models once in a fixed order and then broadcasts the state. The simulation control service receives configuration, geometry, design, workload profile, attitude, mission and constellation commands and writes them into the engine. The data exchange service provides WebSocket broadcast, the REST interface and the renderer polling interface, and all messages share one

envelope. The 3D rendering service is built on Omniverse Kit and provides RTX rendering, USD stages and WebRTC streaming. Each service runs as a process, can be moved to the cloud, and is accessed from a browser.

The application layer builds the whole human-machine interface on the data provided by the service layer, in three groups: constellation configuration and evaluation, compute satellite digital twin, and compute task effectiveness validation. The application layer only projects state into views and controls and performs no physics.

The 3D asset library contains the parametric satellite model generator, six hull architectures, the USD stage library, the geometry parameter store, the offline software renderer and the design thumbnail cache. Geometry commands are written by the service layer into the geometry parameter store, the model generator regenerates the USD file from it, and the 3D rendering service reloads the model after the version number changes.

5.2 System Composition

The platform consists of six subsystems: the simulation model subsystem, the simulation engine subsystem, the data exchange subsystem, the situation display and simulation management subsystem, the 3D rendering subsystem and the 3D asset subsystem. The overall composition is shown in Figure 2.

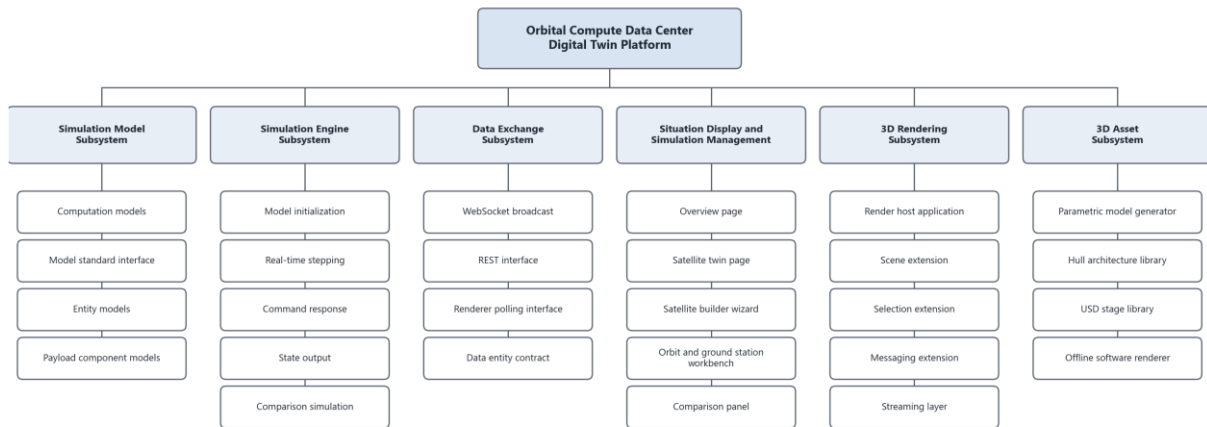


Figure 2 Overall composition of the orbital compute data center digital twin platform

The simulation model subsystem provides all computation models, the model standard interface, the entity models and the payload component models required by the engine. The simulation engine subsystem is responsible for model initialization, real-time stepping, command response, state output and comparison simulation. The data exchange subsystem is responsible for the data interfaces between the browser, the engine and the renderer and for the cross-process data entity contract. The situation display and simulation management subsystem provides the overview page, the satellite twin page, the satellite builder wizard, the orbit and ground station workbench and the comparison panel. The 3D rendering

subsystem implements scene orchestration, component picking, the message channel and streaming as extensions inside the Omniverse Kit host. The 3D asset subsystem provides the parametric model generator, the hull architecture library, the USD stage library and the offline software renderer. The responsibility criterion and code location of each subsystem are given in Table 2.

Table 2 Subsystem responsibilities and code locations

Subsystem	Responsibility criterion	Code location	Technology
Simulation model	Produces business values: position, power, temperature, state of charge, throughput	backend/services, llm_perf.py, ai_workloads.py	Python, sgp4, numpy
Simulation engine	Advances simulation time, integrates state, responds to commands, outputs state	backend/state_engine.py, compare_sim.py	Python, asyncio
Data exchange	Exposes interfaces and broadcast, defines cross-process data entities	backend/app.py, models.py	FastAPI, Pydantic, WebSocket
Situation display and simulation management	Projects state into views and controls, no physics	web/src	React 19, TypeScript, Zustand, Three.js
3D rendering	Projects state onto USD stages and streams it, no business computation	ov_app/exts, ov_app/apps	Omniverse Kit, USD, RTX, WebRTC
3D asset	3D content fully regenerable by scripts	usd, tools	pxr, parametric generator, software rasterizer

The three layers run in three processes, listed in Table 3. The processes are started by one launch script, and the browser connects to the backend WebSocket and to the renderer WebRTC stream automatically after the front-end page opens.

Table 3 Runtime processes and ports

Process	Port	Responsibility
Front-end development server	5173	Serves the React pages; the browser connects to the backend by WebSocket and to the renderer by WebRTC
FastAPI backend	8001	Simulation engine, command response, REST and WebSocket interfaces, USD regeneration pipeline
Omniverse Kit renderer	49100	RTX rendering, USD stages, WebRTC streaming; polls backend state at 5 Hz

5.3 Preliminary Design of the Subsystems

5.3.1 Simulation Model Subsystem

The simulation model subsystem is divided into the computation model module, the model standard interface module, the entity model module and the payload component model

module, as shown in Figure 3. The computation model module provides the basic models for orbit propagation, ground track and coordinate conversion, Sun position and eclipse, ground station visibility, solar array power, battery state of charge, thermal balance, GPU workload and power, LLM inference performance, design margin checking and ground station analytics. The model standard interface module defines four unified interfaces: initialization, step advance, command response and state output. On this basis the compute satellite entity, the constellation entity and the ground station entity were developed, together with five payload components: solar array, battery, radiator, GPU payload and communications.

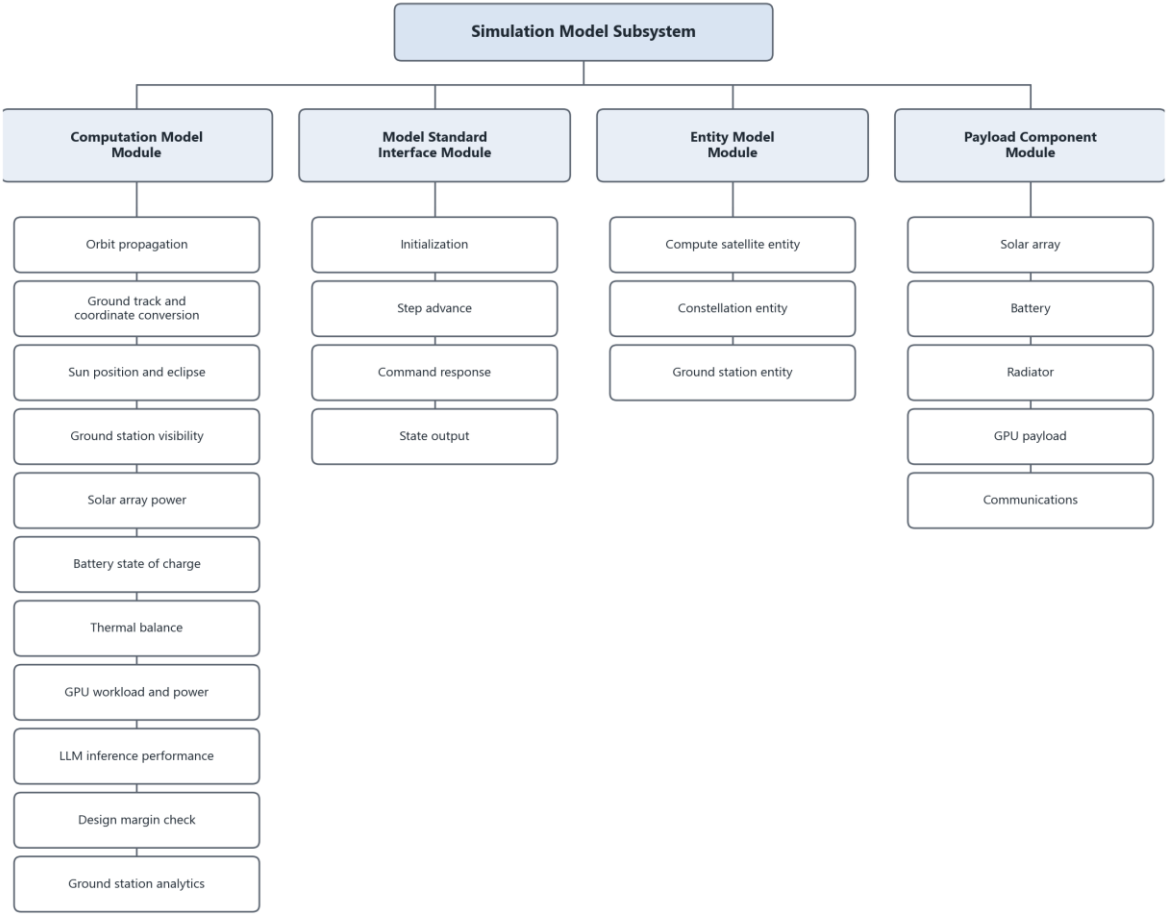


Figure 3 Composition of the simulation model subsystem

Payload components are mounted on the satellite entity and obtain the shared state of orbit position, lighting and structure temperature through the entity. Each component is also an independent model that the engine advances once per physics step. The components form a genuine feedback relationship. The structure temperature given by the radiator component is both the operating temperature of the solar cells and the cold-plate temperature of the GPUs, so it affects generation efficiency and the GPU power budget at the same time.

5.3.2 Simulation Engine Subsystem

The simulation engine subsystem provides the engine kernel for model initialization, real-time stepping, command response, state output and comparison simulation, as shown in Figure 4.

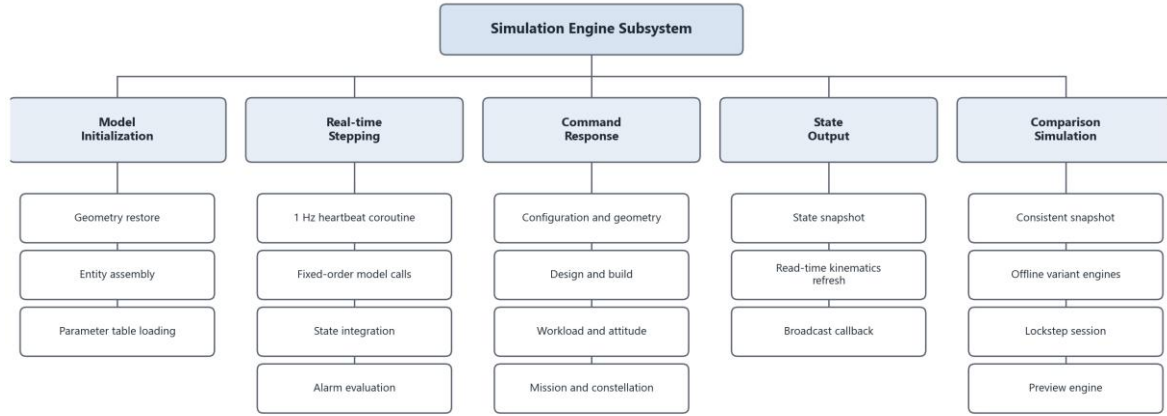


Figure 4 Composition of the simulation engine subsystem

The model initialization module restores geometry parameters from disk at backend start, assembles the entity models and loads the parameter tables. The real-time stepping module advances simulation time with a 1 Hz heartbeat coroutine; every physics step calls the computation models in a fixed order, integrates state of charge, temperature and cumulative output, and then evaluates the alarms. The command response module implements every state-changing operation as a synchronous method called only on the event loop thread, so commands never interleave with a step. The state output module builds the state snapshot and recomputes the geometric quantities from simulation time plus the wall-clock fraction on every read, so that the 5 Hz poller sees sub-second motion. The comparison simulation module freezes a consistent snapshot on the event loop, seeds 2 to 4 offline variant engines from the same snapshot, advances them one step after every physics step, and provides the preview calculation on an independent engine instance for the satellite builder wizard.

5.3.3 Data Exchange Subsystem

The data exchange subsystem provides the network interfaces and the format mapping between the situation display and simulation management subsystem, the simulation engine subsystem and the 3D rendering subsystem, as shown in Figure 5.

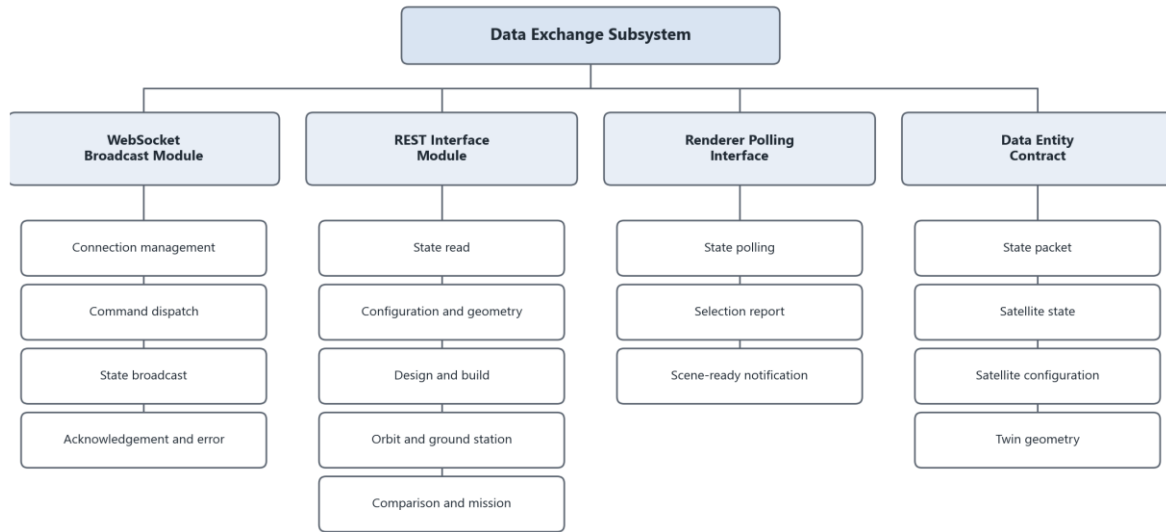


Figure 5 Composition of the data exchange subsystem

The WebSocket broadcast module maintains all browser connections, pushes one state snapshot when a connection is established, broadcasts continuously at 1 Hz afterwards, receives run control and configuration commands from the browser, and returns acknowledgement or error replies. The REST interface module provides state reading, configuration and geometry, design and build, workload profile, orbit and ground station, comparison and mission interfaces. The renderer polling interface lets the 3D rendering subsystem fetch state at 5 Hz and receives component pick and scene-ready events from the renderer. The data entity contract is defined by the backend Pydantic models and mirrored by TypeScript types of the same names in the front end; the top-level broadcast entity is the state packet.

5.3.4 Situation Display and Simulation Management Subsystem

The situation display and simulation management subsystem contains the overview page, the orbit and ground station workbench, the satellite twin page, the satellite builder wizard and the comparison panel. It is the main interface through which the user configures scenarios, observes the situation and analyses data, as shown in Figure 6.

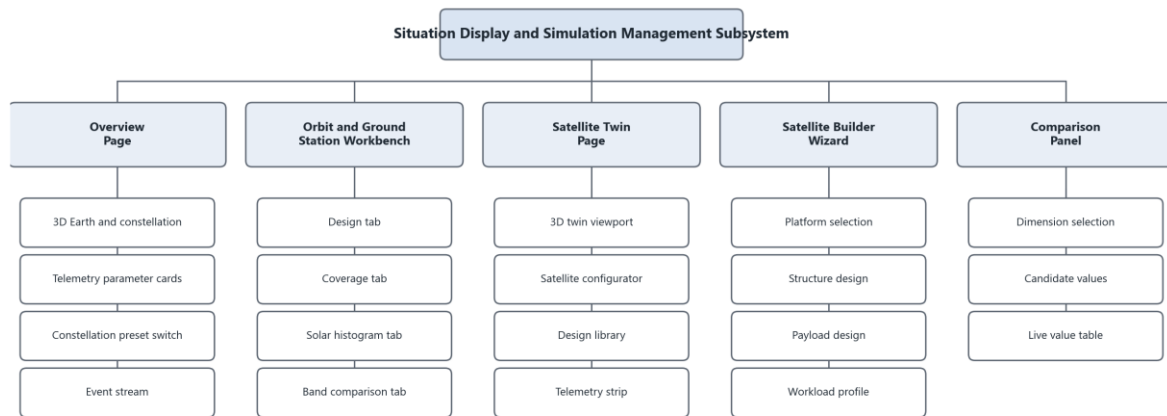


Figure 6 Composition of the situation display and simulation management subsystem

The overview page provides the 3D Earth and constellation, telemetry parameter cards, constellation preset switching and the event stream. The orbit and ground station workbench provides orbital element and Walker parameter design, coverage analysis, the solar histogram and band comparison in four tabs. The satellite twin page provides the 3D twin viewport, the satellite configurator, the design library and the telemetry strip. The satellite builder wizard configures a satellite from scratch in four steps: platform selection, structure design, payload design and workload profile. The comparison panel selects the comparison dimension and candidate values, and after start the variant curves grow in real time on top of the telemetry curves.

5.3.5 3D Rendering Subsystem

The 3D rendering subsystem uses Omniverse Kit as the host application, and all business logic is written in extensions, as shown in Figure 7.

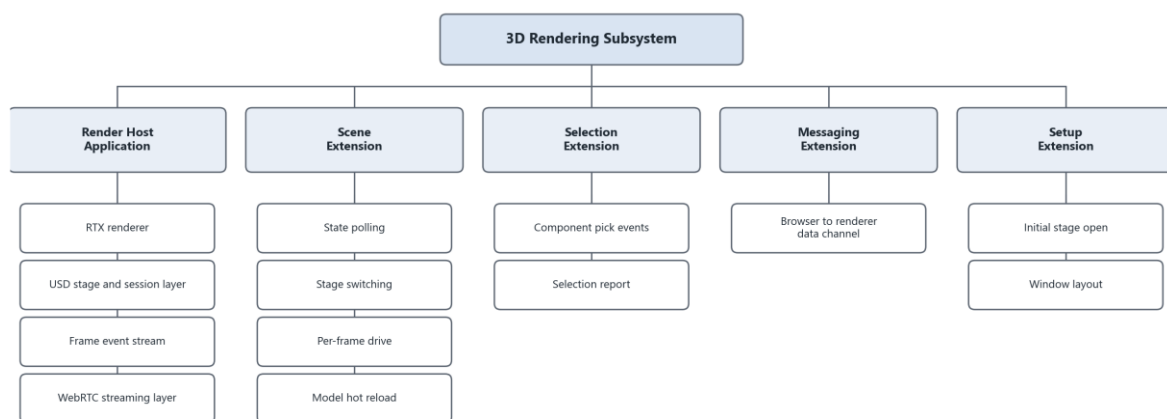


Figure 7 Composition of the 3D rendering subsystem

The render host is defined by the kit configuration file and provides four engine services to the extensions: the frame event stream, the USD stage and context, the RTX renderer and the WebRTC streaming layer. The scene extension is the orchestration hub of the renderer. It

polls the backend at 5 Hz, switches stages, drives the Earth, the Sun, the lights, the satellite attitude, the roll-out wing and the constellation point cloud every frame, and hot-reloads the geometry model. The selection extension subscribes to USD pick events and reports them to the backend. The messaging extension provides the control message channel between the browser and the renderer. The setup extension opens the initial stage and lays out the window.

5.3.6 3D Asset Subsystem

The 3D asset subsystem provides 3D content that scripts can regenerate completely, as shown in Figure 8.

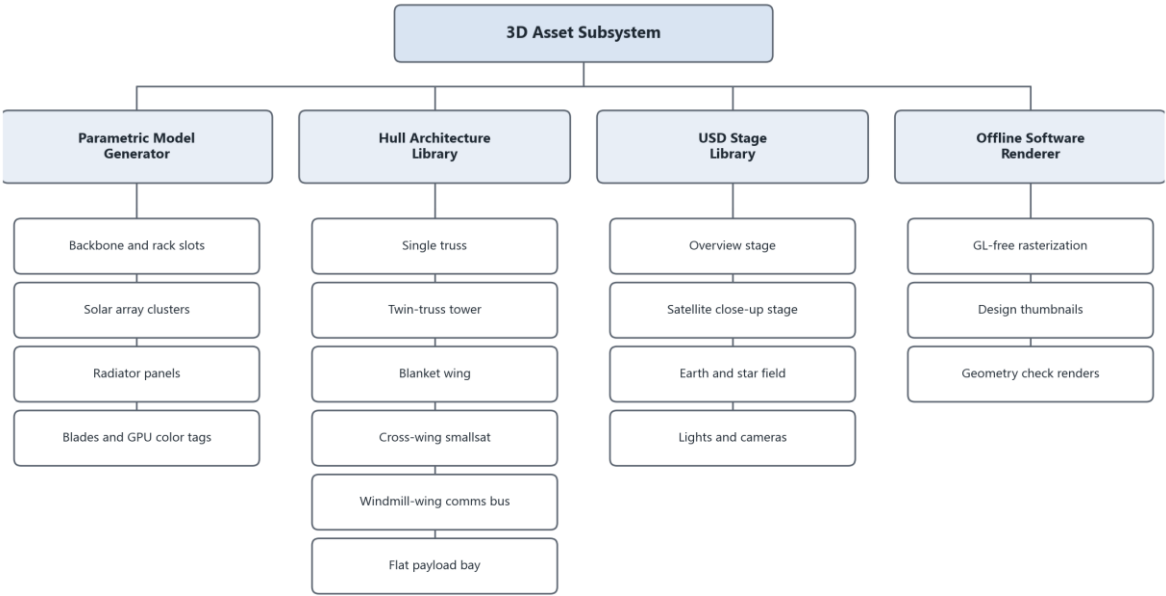


Figure 8 Composition of the 3D asset subsystem

The parametric model generator reads the geometry parameter file and generates the backbone, the rack slots, the solar array clusters, the radiator panels and the server blades, and attaches a GPU color tag to every blade that carries a card. The hull architecture library contains six architectures: single truss, twin-truss tower, blanket wing, cross-wing smallsat, windmill-wing communications bus and flat payload bay. The USD stage library contains the overview stage, the satellite close-up stage, and the Earth, star field, lights and cameras. The offline software renderer rasterizes any stage without starting the renderer and produces design thumbnails and geometry check renders.

5.4 Usage Flow

The usage flow of the platform is shown in Figure 9. The user first edits the scenario in the situation display and simulation management subsystem. The scenario comprises the satellite platform, the structure and payload configuration, the workload profile, the orbital elements

and Walker parameters, the ground station and communication band, and the attitude mode. Edits enter the data exchange subsystem as REST or WebSocket commands and reach the command response methods of the simulation engine. Geometry commands additionally make the 3D asset subsystem regenerate the USD model; the version number increases after the file is written, and the renderer reloads the model in its next polling period.

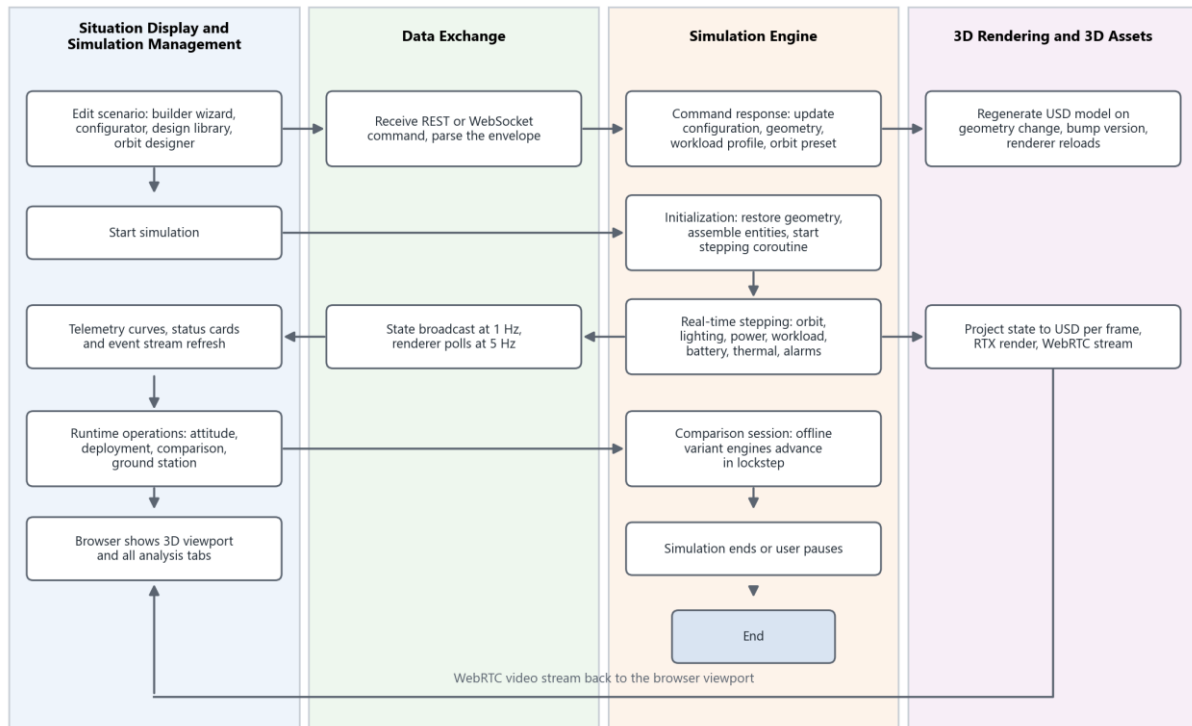


Figure 9 Usage flow of the orbital compute data center digital twin platform

After the user starts the simulation the engine steps at 1 Hz. Every physics step performs orbit propagation, lighting determination, solar array power, workload operating point solution, battery integration, thermal balance integration, downlink determination and alarm evaluation in turn. The state data goes one way over WebSocket at 1 Hz to the telemetry curves and status cards of the browser, and another way to the renderer, which fetches it at 5 Hz, projects it onto the USD stage every frame, renders it with RTX and streams it over WebRTC back to the browser viewport. During the run the user can switch the attitude mode, deploy or retract the solar array and configure the ground station, and can start a comparison session in which offline variant engines advance in lockstep with the live simulation until the user pauses or ends the run.

6 Detailed Design

6.1 Simulation Model Subsystem

6.1.1 Model Hierarchy

The simulation models are organized from top to bottom into concept models, algorithm models and entity models, as shown in Figure 10. Concept models describe the common technical framework of three kinds of objects, the compute satellite, the satellite constellation and the ground station, including overall parameters and interaction relations. Algorithm models are computation methods shared by the object kinds and can be called by several entity models. Entity models instantiate concrete objects within the concept model framework and connect to the engine through the model standard interface.

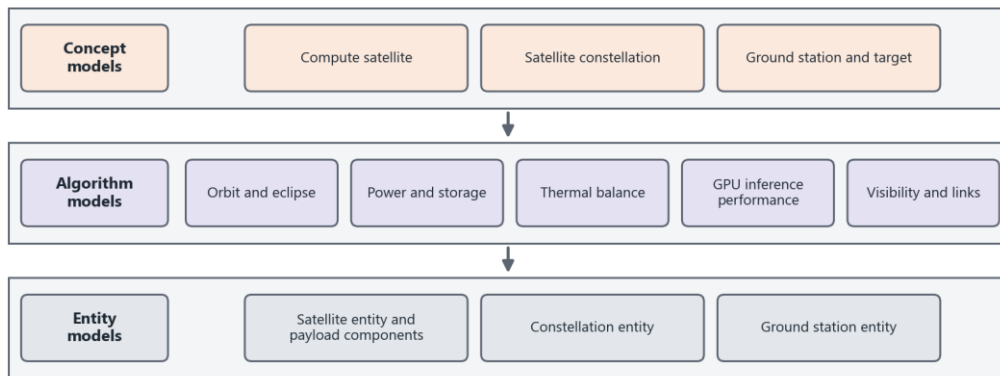


Figure 10 Simulation model hierarchy

6.1.2 Concept Models

Compute satellite: a compute satellite is a low Earth orbit satellite whose main business is GPU inference serving. It consists of the satellite platform, the solar array, the battery, the radiator, the GPU payload and the communications payload. The satellite moves along its orbit and periodically enters and leaves eclipse. The solar array generates power in sunlight and the battery supplies power in eclipse. The GPU payload runs according to the workload schedule and produces waste heat. The radiator emits the waste heat to deep space while absorbing solar, Earth albedo and Earth infrared flux. The structure temperature determines the GPU cold-plate temperature; when the cold plate is too hot the GPUs throttle and the inference throughput falls. Its main functions are:

1. Orbit motion and lighting: propagate the orbit from the TLE and give the ground track, the sunlight state and the Sun vector.
2. Generation and storage: compute generated power from area, material, attitude, Sun distance and temperature; compute battery capacity from chemistry and pack size and integrate the state of charge.

3. Compute payload operation: determine the current job from the workload profile and solve frequency, power and throughput from the GPU model and the thermal constraint.
4. Thermal balance: integrate the structure temperature as a lumped thermal node and give radiated power, die temperature and throttling state.
5. Design check and alarms: give the average supply-demand margin, the heat rejection margin, the eclipse depth of discharge and eight kinds of alarms.

Satellite constellation: a constellation consists of several orbital planes with several satellites per plane in a Walker pattern, and all satellites share the current satellite design. The constellation as a whole gives the number in orbit, the number in sunlight, the aggregate generated power, and the numbers of inter-satellite and ground links. Its main functions are:

1. Constellation generation: synthesize the reference TLE from the orbital elements and Walker parameters and derive the TLEs of all members.
2. Fleet propagation: propagate the position and velocity of every member each physics step and give ground track and sunlight state.
3. Aggregate metrics: give aggregate generated power, eclipse count, link counts and coverage fraction.

Ground station and target: a ground station is defined by geodetic latitude, longitude and altitude and carries an elevation mask and a communication band. Its main functions are:

1. Visibility determination: compute the elevation of every satellite on the WGS-84 ellipsoid and test it against the mask.
2. Access windows and downlink: merge samples into access windows and downlink at the band rate while visible.
3. Aggregate analytics: give the visible satellite count, aggregate bandwidth, elevation distribution and solar collection histogram.

6.1.3 Algorithm Models

Following component-based and parametric modelling, the functions of the concept models are decomposed into algorithms. Orbit motion and lighting decompose into the orbit propagation, ground track and coordinate conversion, and Sun position and eclipse models. Generation and storage decompose into the solar array power and battery state of charge models. Compute payload operation decomposes into the GPU workload and power model and the LLM inference performance model. Thermal balance corresponds to the thermal balance model. Design check and alarms correspond to the design margin check model. Ground station visibility, access windows and aggregate analytics decompose into the ground station visibility and ground station analytics models. The resulting algorithm model list is given in Table 4.

Table 4 Complete set of algorithm models

No.	Function	Identifier	Description
1	Orbit propagation	Propagate	Propagates the satellite position and velocity in the TEME frame with SGP4 from a TLE; supports TLE synthesis from orbital elements
2	Ground track and coordinate conversion	GroundTrack	Computes WGS-84 geodetic latitude, longitude and altitude from the TEME position; converts between orbital elements and state vectors
3	Sun position and eclipse	SunEclipse	Analytic Sun position, Sun distance, solar irradiance and the conical umbra and penumbra eclipse fraction
4	Ground station visibility	Access	Computes the elevation of the satellite from the ground station, tests visibility and merges access windows
5	Solar array power	SolarPower	Computes generated power from area, efficiency, attitude incidence, eclipse fraction, temperature derating and deployment
6	GPU workload and power	GpuWorkload	Solves the job operating point, power and throughput per GPU group from the workload profile
7	LLM inference performance	LlmPerf	Solves frequency, throughput and die temperature from the DVFS power law and the memory-bound decode law
8	Battery state of charge	Battery	Derives capacity from chemistry and pack size and integrates the state of charge
9	Thermal balance	Thermal	Lumped thermal node integration with environmental flux and radiator emission
10	Design margin check	DesignCheck	Average supply-demand margin, heat rejection margin and alarm evaluation
11	Ground station analytics	GroundAnalytics	Visible satellite count, aggregate bandwidth, elevation distribution and solar collection histogram

Model 1 Orbit propagation model**Table 5 Orbit propagation model**

Name	Orbit propagation
Interface	Propagate
Summary	Propagates the satellite orbit over the scenario simulation period
Called by	Satellite entity, constellation entity, comparison simulation module, renderer read-time refresh
Calls	None
Input	Form 1: two-line element set and simulation time. Form 2: semi-major axis, eccentricity, inclination, right ascension of the ascending node, argument of perigee, mean anomaly and Walker parameters; the platform synthesizes a TLE and then proceeds as in form 1.
Method	The SGP4 analytic propagator is used. Simulation time is converted by the time base module into minutes since the TLE epoch, and SGP4 returns the position and velocity in the TEME frame.

	When the input is a set of orbital elements the mean motion follows from the semi-major axis by Kepler's third law, $n = 86400 / 2\pi \cdot \sqrt{(\mu / a^3)}$ where μ is the Earth gravitational constant. The mean motion and the other five elements are written into the second TLE line in the CCSDS two-line format. The orbital planes of a Walker constellation are spaced equally in right ascension, the members within a plane are spaced equally in mean anomaly with the phasing factor, and all members share the first TLE line. The SGP4 object of the tracked satellite is cached separately so that a read-time refresh costs one propagation.
Output	Form 1: TEME position and velocity at each time node. Form 2: osculating orbital elements at each time node.

Model 2 Ground track and coordinate conversion model

Table 6 Ground track and coordinate conversion model

Name	Ground track and coordinate conversion
Interface	GroundTrack
Summary	Computes WGS-84 geodetic latitude, longitude and altitude from the TEME position and converts between orbital elements and state vectors
Called by	Satellite entity, constellation entity, ground station visibility model
Calls	None
Input	TEME position vector and the corresponding Julian date; or orbital elements; or a position and velocity vector
Method	First, Greenwich mean sidereal time is computed by the IAU-82 formula, $\theta_g = 280.46061837 + 360.98564736629 \cdot (\text{JD} - 2451545.0) + 0.000387933 \cdot T^2 - T^3 / 38710000$ where T is the number of Julian centuries since J2000.0. The position vector is rotated about the z axis by θ_g to obtain Earth-fixed coordinates. Second, geodetic latitude ϕ and altitude h are solved iteratively on the WGS-84 ellipsoid, $N = a / \sqrt{1 - e^2 \sin^2 \phi}, \quad p = (N + h) \cos \phi, \quad z = (N(1 - e^2) + h) \sin \phi$ where a is the equatorial radius, e the first eccentricity and p the projected length of the position in the equatorial plane. Longitude follows directly from the x and y components. Third, the conversion between orbital elements and state vectors follows the classical two-body relations; singular cases follow the conventions of Document 1, and the osculating elements are broadcast every physics step.
Output	Geodetic longitude, latitude and altitude; or position and velocity; or osculating orbital elements

Model 3 Sun position and eclipse model

Table 7 Sun position and eclipse model

Name	Sun position and eclipse
Interface	SunEclipse
Summary	Computes the Sun direction, the Sun distance, the solar irradiance and the visible

	fraction of the solar disc seen from the satellite
Called by	Satellite entity, constellation entity, solar array power model, thermal balance model, renderer lighting
Calls	None
Input	Julian date; satellite position vector
Method	First, the Sun position is computed by the low-precision analytic model of Document 4. The mean longitude, mean anomaly, ecliptic longitude and obliquity are $\lambda_M = 280.460 + 36000.771 \cdot T$, $M = 357.5291092 + 35999.05034 \cdot T$ $\lambda = \lambda_M + 1.914666471 \cdot \sin M + 0.019994643 \cdot \sin 2M$, $\varepsilon = 23.439291 - 0.0130042 \cdot T$ The Sun distance d in astronomical units is $d = 1.000140612 - 0.016708617 \cdot \cos M - 0.000139589 \cdot \cos 2M$ and the Sun unit vector is converted from λ and ε into the equatorial frame. Second, the solar irradiance is corrected for Sun distance, $S = S_0 / d^2$, with $S_0 = 1361$ W/m². Third, the visible solar disc fraction is computed with the conical umbra and penumbra model. The apparent radii of the Sun and the Earth are $\rho_s = \arcsin(R_s / r_s - r)$, $\rho_e = \arcsin(R_e / r)$ Let ψ be the angle between the Sun direction and the Earth direction. For $\psi \geq \rho_e + \rho_s$ the satellite is in full sunlight and the fraction is 1; for $\psi \leq \rho_e - \rho_s$ it is in umbra and the fraction is 0; otherwise it is in penumbra and the fraction is the unoccluded area of the solar disc divided by the disc area, computed with the formula for the intersection of two circles.
Output	Sun unit vector; Sun distance; solar irradiance; visible solar disc fraction; sunlit flag

Model 4 Ground station visibility model

Table 8 Ground station visibility model

Name	Ground station visibility
Interface	Access
Summary	Determines visibility between a satellite and a ground station and generates access windows
Called by	Satellite entity, ground station entity, ground station analytics model, downlink determination
Calls	GroundTrack
Input	Satellite Earth-fixed position; ground station geodetic longitude, latitude and altitude; elevation mask
Method	First, the ground station geodetic coordinates are converted to Earth-fixed coordinates on the WGS-84 ellipsoid. Second, the line-of-sight vector from the station to the satellite is projected onto the east-north-up frame of the station, and the elevation is $el = \arcsin(\hat{p} \cdot \hat{u})$ where $\hat{p}$ is the line-of-sight unit vector and $\hat{u}$ the ellipsoid normal at the station. Third, the satellite is visible when the elevation is not below the mask. The effective mask is the larger of the user setting and the minimum elevation of the communication band. Fourth, offline analysis samples one full orbit at a fixed step and merges consecutive

	visible samples into access windows, giving window edges, coverage fraction and the countdown to the next pass.
Output	Visible flag; elevation; access window list

Model 5 Solar array power model

Table 9 Solar array power model

Name	Solar array power
Interface	SolarPower
Summary	Computes the generated power of the solar array at the current instant
Called by	Satellite entity, constellation entity, design margin check model, ground station analytics model
Calls	SunEclipse
Input	Array area; cell material; attitude mode; position and velocity vectors; Sun unit vector; solar irradiance; visible solar disc fraction; structure temperature; deployment fraction
Method	The generated power is $P_{\text{solar}} = \eta \cdot A \cdot S \cdot k_{\text{inc}} \cdot f_{\text{illum}} \cdot \eta_T \cdot k_{\text{dep}}$ where η is the cell efficiency, A the array area, S the solar irradiance at the current Sun distance, k_{inc} the incidence factor, f_{illum} the visible solar disc fraction, η_T the temperature derating factor and k_{dep} the deployment fraction. The incidence factor depends on the attitude mode. In Sun-pointing mode the array drive keeps the wings facing the Sun and the incidence is 1. Free mode is a Sun-tracking wing with incidence 0.95, or 1 in a dawn-dusk Sun-synchronous orbit. In the three body-fixed modes, nadir, ram and inertial, the wing normal is the position unit vector, the velocity unit vector and the orbit normal unit vector respectively, and the incidence is the non-negative part of the dot product of the normal with the Sun vector. The temperature derating factor follows the linear datasheet relation $\eta_T = 1 + k_T \cdot (T_{\text{array}} - 25)$ where k_T is the power temperature coefficient of the material and T_{array} the equilibrium temperature of the array. The array is thermally decoupled from the bus and its equilibrium temperature is solved from a two-sided radiation balance, $(\alpha - \eta_e) \cdot S \cdot k_{\text{inc}} \cdot f_{\text{illum}} + \alpha \cdot q_{\text{IR}} \cdot F = (\epsilon_f + \epsilon_b) \cdot \sigma \cdot T_{\text{array}}^4$ where α is the cell absorptance, η_e the fraction converted to electricity, q_{IR} the Earth infrared flux, F the view factor of a sphere to the Earth, and ϵ_f and ϵ_b the front and back emittances. When a roll-out array deploys or retracts, the deployment fraction moves linearly toward the target over 12 s.
Output	Generated power; incidence factor; array temperature

Model 6 GPU workload and power model

Table 10 GPU workload and power model

Name	GPU workload and power
Interface	GpuWorkload

Summary	Determines the current job from the workload profile and solves the operating point, power and throughput per GPU group
Called by	Satellite entity, design margin check model, workload profile fit assessment
Calls	LlmPerf
Input	Workload profile; simulation time; GPU groups, that is each card type with its count; structure temperature; sunlight and state of charge
Method	First, a workload profile is a cyclic table of job blocks, each giving start time, duration, duty fraction and job type. The current block is found by taking the simulation time modulo the cycle. In eclipse with a state of charge below 0.4 the duty is limited to 0.2 and the job degrades to housekeeping. Second, the payload bay is grouped by card type. Each group is solved as one tensor-parallel group; the satellite total is the sum over groups, per-card metrics are weighted by card count, the die temperature is the maximum over groups and the throttle flag is the logical or. A homogeneous bay reduces to a single group. Third, LLM jobs call the LLM inference performance model for frequency, realized draw and throughput. Vision and housekeeping jobs are estimated from datasheet peak throughput and model utilization, with per-card power $P_{\text{card}} = \text{TDP} \cdot (0.15 + 0.85 \cdot u)$ where u is the duty fraction of the job block. Vision throughput is $\text{frames/s} = \text{MFU} \cdot \text{OPS}_{\text{peak}} / 0.30 \text{ TFLOP}$ and training throughput is $\text{tok/s} = \text{MFU} \cdot \text{OPS}_{\text{peak}} \cdot N_{\text{card}} / (6 \cdot N_{\text{params}})$ Fourth, the payload power is the sum of the realized draw over groups. Cumulative output is integrated 1:1 in simulation time and reset when the workload profile or the design changes.
Output	Current job and model; frequency, power, die temperature and throttle flag per group; satellite throughput; payload power; cumulative output

Model 7 LLM inference performance model

Table 11 LLM inference performance model

Name	LLM inference performance
Interface	LlmPerf
Summary	Solves GPU frequency, throughput, realized draw and die temperature under power and thermal constraints
Called by	GpuWorkload
Calls	None
Input	GPU model; model size and layer parameters; precision; batch size; context length; power cap; structure temperature; tensor-parallel group size
Method	First, the power-frequency relation is the DVFS aggregate $P(x) = P_{\text{static}} + \chi \cdot x^\theta, \quad x = f_{\text{sm}} / f_{\text{max}}$ For a given power budget P the frequency ratio is $x(P) = ((P - P_{\text{static}}) / \chi)^{1/\theta}$, limited to at most 1. Second, the prefill and training phases are compute-bound and the throughput is a single power law, $\text{tok/s} = T_{\text{fmax}} \cdot x^p, \quad T_{\text{fmax}} = \eta_{\text{pre}} \cdot \text{OPS}_{\text{peak}} / (k \cdot N_{\text{params}})$

	where k is 2 for inference and 6 for training. Third, the decode phase is memory-bound; every step re-reads all weights and the KV cache of the batch, and the throughput is $\text{tok/s} = B / (T_{\text{mem}} + C \cdot (x^p - 1))$ $T_{\text{mem}} = (W_{\text{bytes}} + B \cdot C_{\text{eff}} \cdot k v_{\text{bytes}}) / BW_{\text{eff}}, \quad C = k \cdot N_{\text{params}} \cdot B / (\eta_{\text{dec}} \cdot \text{OPS}_{\text{peak}})$ where B is the batch size, T_{mem} the memory floor time and C the compute time at full frequency. Weights, KV cache and compute are sharded over the cards of a tensor-parallel group, the aggregate throughput equals the single-card full-model solution times the card count, and the batch is clamped when the memory cannot hold the weights and the KV cache. Fourth, the realized draw in decode is below the power cap, $P_{\text{nat}} = P_{\text{static}} + \chi \cdot (0.70 + 0.30 \cdot \text{duty}), \quad \text{duty} = C / (T_{\text{mem}} + C)$ Fifth, the die temperature couples to the structure temperature through the conduction stack, $T_{\text{die}} = T_{\text{struct}} + P \cdot R_{\text{th}}$ The driver keeps the die below the throttle target, which is equivalent to shrinking the power budget, $P_{\text{limit}} = (T_{\text{throttle}} - T_{\text{struct}}) / R_{\text{th}}, \quad P_{\text{eff}} = \min(P_{\text{cap}}, P_{\text{limit}})$ A P_{limit} below the static power is flagged as thermal runaway. The parameters are calibrated against the V100 power-cap sweep of Document 3; A100, H200 and B200 use datasheet peaks and serving-stack attainment fractions.
Output	Frequency ratio; throughput; realized draw; die temperature; throttle and runaway flags; throttle onset structure temperature

Model 8 Battery state of charge model

Table 12 Battery state of charge model

Name	Battery state of charge
Interface	Battery
Summary	Derives the battery capacity and integrates the state of charge
Called by	Satellite entity, design margin check model
Calls	None
Input	Battery chemistry; pack size; generated power; payload power; platform power; time step
Method	The capacity follows from the pack mass and the gravimetric energy density of the chemistry, $E_{\text{batt}} = m_{\text{pack}} \cdot \rho_E$ The net electrical power is the generated power minus payload and platform power, $P_{\text{net}} = P_{\text{solar}} - P_{\text{payload}} - P_{\text{platform}}$ The round-trip loss is booked once, on the charging leg: only the fraction η_{rt} of a surplus reaches storage, and discharge draws 1:1, $P_{\text{eff}} = \eta_{\text{rt}} \cdot P_{\text{net}} \quad \text{for } P_{\text{net}} \geq 0; \quad P_{\text{eff}} = P_{\text{net}} \quad \text{for } P_{\text{net}} < 0$ The state of charge is integrated as energy and clamped to the range 0 to 1, $\Delta \text{SOC} = P_{\text{eff}} \cdot \Delta t \cdot k_{\text{time}} / (3600 \cdot E_{\text{batt}})$ where k_{time} is the time acceleration of the battery and thermal dynamics, equal to 60.
Output	Battery capacity; net power; state of charge

Model 9 Thermal balance model

Table 13 Thermal balance model

Name	Thermal balance
Interface	Thermal
Summary	Integrates the structure temperature as a lumped thermal node and gives the radiated power
Called by	Satellite entity, design margin check model, LLM inference performance model
Calls	SunEclipse
Input	Payload power; platform power; radiator area; coating emittance ε and absorptance α ; position vector; solar irradiance; visible solar disc fraction; cosine of the Sun to position angle; time step
Method	The heat input is the sum of electrical dissipation and environmental flux; 95% of the electrical power becomes heat and the remainder leaves as radio frequency, $Q_{in} = 0.95 \cdot (P_{payload} + P_{platform}) + q_{env}$ The environmental flux has three terms: direct solar absorption, Earth albedo absorption and Earth infrared absorption, $q_{env} = \alpha \cdot S \cdot f_{illum} \cdot A / 4 + \alpha \cdot a_E \cdot S \cdot A \cdot F \cdot \max(0, \hat{r} \cdot \hat{s}) + \varepsilon \cdot q_{IR} \cdot A \cdot F$ where a_E is the Earth albedo, 0.30, q_{IR} the Earth infrared flux, 237 W/m², and F the view factor of a sphere to the Earth, $F = (1 - \sqrt{(1 - (R_E / r)^2)}) / 2$ The heat output is grey-body emission from both faces of the radiator panels, $Q_{out} = \varepsilon \cdot \sigma \cdot A \cdot T^4$ The structure temperature is integrated with the lumped heat capacity, $dT/dt = (Q_{in} - Q_{out}) / C_{th}$ C_{th} is 160 kJ/K and the temperature is limited to the range -80 °C to 95 °C. The radiator area is derived from the geometry parameters; two panels radiate from both faces, and the flat payload bay architecture has one panel with two faces.
Output	Structure temperature; radiated power; environmental flux

Model 10 Design margin check model

Table 14 Design margin check model

Name	Design margin check
Interface	DesignCheck
Summary	Gives the steady-state sizing margins and evaluates the alarms
Called by	Satellite entity, workload profile fit assessment, satellite builder preview
Calls	SolarPower, GpuWorkload, Thermal
Input	Current configuration and geometry; workload profile; orbit type; current state
Method	The average demand is the duration-weighted mean of the realized draw of the job blocks of the workload profile plus the platform power. The average supply is $P_{supply} = \eta \cdot A \cdot S_o \cdot \bar{k}_{inc} \cdot 0.5$ where $\bar{k}_{inc}$ is the orbit-average incidence of the current attitude mode and 0.5 the sunlit fraction; a dawn-dusk Sun-synchronous orbit takes full sunlight.

	The peak thermal demand is the dissipation at sustained full load, $Q_{\text{peak}} = 0.95 \cdot (\Sigma \text{TDP} + P_{\text{platform}})$ The maximum heat rejection is the emission at the 60 °C ceiling minus the worst-case environmental flux, $Q_{\text{emit,max}} = \varepsilon \cdot \sigma \cdot A \cdot (333.15 \text{ K})^4 - q_{\text{env,worst}}$ Supply minus demand gives the power margin, and rejection minus peak demand gives the thermal margin. The eclipse depth of discharge is estimated for the worst-aligned eclipse window. The design check and the per-step integration use the same formulas, so a design that passes the check closes in the running simulation. The alarm conditions are listed in Table 15.
Output	Average supply and demand; peak thermal demand and maximum rejection; margins; alarm list

Table 15 Alarm conditions

Alarm	Condition
low_battery	State of charge below 0.20
overtemp	Structure temperature above 70 °C
undertemp	Structure temperature below −40 °C
eclipse_deficit	In eclipse with state of charge below 0.35 and negative net power
radiator_undersized	Maximum heat rejection below 90% of the peak thermal demand
solar_undersized	Average supply below average demand
gpu_thermal_throttle	GPU die held at the throttle target; the power budget is cut by the thermal constraint
gpu_thermal_runaway	Thermally constrained power budget below the static power; the GPU cannot even idle within budget

Model 11 Ground station analytics model

Table 16 Ground station analytics model

Name	Ground station analytics
Interface	GroundAnalytics
Summary	Computes the ground station aggregate metrics inside the same loop as the fleet propagation
Called by	Constellation entity, ground station entity
Calls	Access, SolarPower
Input	Earth-fixed positions and geodetic coordinates of all members; ground station; effective elevation mask; band rate per satellite; peak generation per satellite; solar histogram bin count
Method	The elevation of every satellite is computed and tested for visibility; the visible count and the best elevation are output as live metrics. The aggregate bandwidth is the visible count times the band rate per satellite. The elevation distribution counts visible satellites against a fixed mask sequence from 0° to 40°, giving the cumulative elevation distribution. The solar histogram uses 100 times the non-negative dot product of the position vector and the Sun vector as the illumination intensity, bins it into 5 or 10 bins, and assigns each bin the collected power as intensity times the peak

	generation per satellite, so that the bins sum to the constellation aggregate generation. The index list of visible members is broadcast with the state for the coverage map.
Output	Visible satellite count; best elevation; aggregate bandwidth; cumulative elevation distribution; solar collection histogram; visible member indices

6.1.4 Parameter Catalogs

The hardware and material parameters used by the algorithm models are kept in parameter catalogs; changing a catalog entry changes the corresponding design variable. The GPU catalog is given in Table 17. The V100 parameters are the fitted values of Document 3, and the other models use datasheet peaks and serving-stack attainment fractions.

Table 17 GPU catalog

Model	TDP W	Max clock MHz	FP16 dense peak TFLOPS	FP8 dense peak TFLOPS	Memory bandwidth TB/s	Static power W	Junction to structure K/W	Throttle target °C
V100	250	1530	125	125	0.90	50	0.120	83
A100	400	1410	312	312	2.04	80	0.085	85
H200	700	1980	989	1979	4.80	145	0.060	85
B200	1000	1965	2250	4500	8.00	200	0.045	85

Table 18 Solar cell catalog

Material	Efficiency	Power temperature coefficient %/K	Areal density kg/m ²
Si	0.22	−0.45	2.5
GaAs	0.32	−0.20	3.0
Perovskite	0.38	−0.30	1.8

Table 19 Radiator coating catalog

Coating	Emittance ϵ	Absorptance α	Areal density kg/m ²
Aluminum	0.10	0.25	4.0
WhitePaint	0.85	0.25	4.4
OSR	0.92	0.08	4.6
Graphite	0.96	0.90	3.6

Table 20 Battery chemistry and pack size catalog

Chemistry	Energy density Wh/kg	Round-trip efficiency	Pack size	Mass kg
LiIon	250	0.95	S	10
LiFePO4	160	0.96	M	20
LiS	400	0.90	L	32

Chemistry	Energy density Wh/kg	Round-trip efficiency	Pack size	Mass kg
SolidState	350	0.97	XL	60

Table 21 LLM model catalog

Identifier	Model	Parameters	Layers	KV heads	Head dimension
llama8b	Llama-3.1-8B	8×10^9	32	8	128
mistral24b	Mistral-Small-24B	24×10^9	40	8	128
qwen32b	Qwen2.5-Coder-32B	32.8×10^9	64	8	128
llama70b	Llama-3.3-70B	70×10^9	80	8	128
qwen72b	Qwen2.5-72B	72.7×10^9	80	8	128
llama405b	Llama-3.1-405B	405×10^9	126	8	128

Table 22 Job type catalog

Job identifier	Job	Model	Precision	Phase	Batch	Context
housekeeping	Housekeeping	None	None	Idle		
vision_batch	EO imagery batch inference	ViT-L/16	FP8	Vision		
vision_burst	EO target burst classification	ViT-L/16	FP8	Vision		
llm_batch	LLM batched inference	Llama-3.3-70B	FP8	Decode	48	2048
llm_interactive	LLM interactive serving	Llama-3.3-70B	FP8	Decode	8	1024
llm_pretrain	LLM pretraining	Llama-3.3-70B	BF16	Training		8192
llm_finetime	LLM adapter fine-tuning	Llama-3.1-8B	BF16	Training		4096
llm_eval	LLM evaluation pass	Llama-3.3-70B	FP8	Decode	24	4096
llm_chat_70b	Chat serving 70B	Llama-3.3-70B	FP8	Decode	24	4096
llm_chat_8b	Edge chat 8B	Llama-3.1-8B	FP8	Decode	64	2048
llm_code_32b	Code assist	Qwen2.5-Coder-32B	FP8	Decode	16	8192
llm_rag_72b	Long-context RAG	Qwen2.5-72B	FP8	Decode	8	16384
llm_summarize_24b	Document summarization	Mistral-Small-24B	FP8	Decode	32	8192
llm_frontier_405b	Frontier serving 405B	Llama-3.1-405B	FP8	Decode	12	4096
checkpoint_io	Checkpoint write	None	None	Idle		

Table 23 Workload profile catalog

Profile identifier	Name	Main jobs
inference	LLM serving 70B, default profile	Interactive serving alternating with batched decode
chat_serving	Multi-tier chat serving	70B quality tier and 8B edge tier

Profile identifier	Name	Main jobs
frontier	Frontier serving 405B	Long 405B serving after an evaluation pass
code_rag	Code and RAG serving	Code assist, document summarization, long-context RAG
balanced	Mixed inference	Interactive serving, code assist, batched inference, burst classification, edge chat
training	Sustained training	Pretraining epochs with checkpoint writes
burst	Burst response	Burst classification alternating with long housekeeping
low_duty	Low duty cycle	Mostly housekeeping with scheduled batch vision inference

6.1.5 Entity Models

Model 1 Compute satellite entity model

The satellite entity is defined with the compute satellite concept model. Platform parameters, slot payloads and geometry parameters come from the satellite builder wizard or the design library.

Table 24 Compute satellite entity model

Entity name	Compute satellite entity
Entity class	ComputeSatellite
Composition	Single platform with several components
Components	Orbit component; solar array component; battery component; radiator component; GPU payload component; communications component
Interfaces	Propagate orbit component SolarPower solar array component Battery battery component Thermal radiator component GpuWorkload and LlmPerf GPU payload component Access communications component
Entity parameters	Platform architecture and slot count; GPU model per slot; cell material and wing segment count; coating, radiator span and aspect ratio; battery chemistry and pack size; platform power; workload profile; attitude mode; orbit TLE
State output	Latitude, longitude and altitude; sunlight and Sun angle; generated, payload and platform power; state of charge; structure temperature and radiated power; job detail and die temperature; deployment fraction; attitude; osculating elements; alarm list

The run flow of the compute satellite entity is shown in Figure 11. After initialization every physics step performs orbit propagation and lighting determination, solves generation and the workload operating point, integrates the battery and the thermal node, shrinks the power budget and records the throttle alarm when the die temperature exceeds the throttle target, and then writes back the state and advances to the next step until the simulation ends.

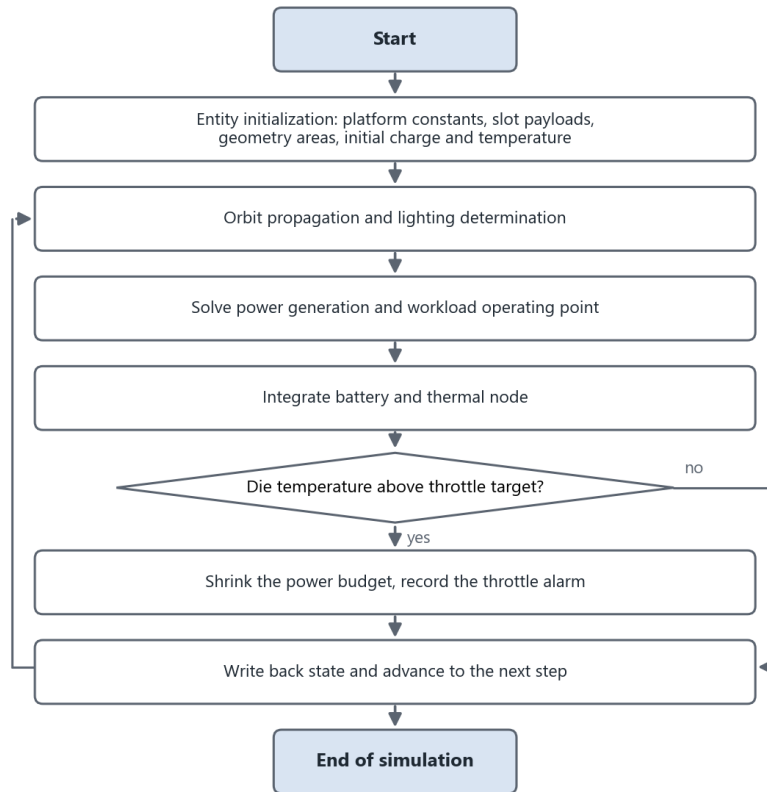


Figure 11 Run flow of the compute satellite entity

The satellite platforms available to the entity are listed in Table 25. Each platform maps to one hull architecture and carries a slot count, and its factory configuration is a design preset already verified for that architecture.

Table 25 Satellite platform catalog

Identifier	Platform	Architecture	Slots	Factory payload	Description
spacex	SpaceX Compute Bus	Single truss	12	8×H200	Vertical truss, four quadrant racks, silicon wings
redwire	Redwire Payload Bay	Flat payload bay	7	7×H200	Bay carries 7 GPU modules, roll-out GaAs wing, single radiator
sophia	Sophia Space TILE	Cross-wing smallsat	6	4×H200	Flat tile with solar face and radiating back; viewport shows a stand-in hull
ada	Ada Space Compute Node	Windmill-wing comms bus	8	8×H200	Networked twin-wing smallsat with laser inter-satellite links; viewport shows a stand-in hull

The design library provides six complete satellite design presets, listed in Table 26. Each preset contains the hardware configuration, the geometry, the workload profile and the platform power, and applying a preset switches the four together atomically.

Table 26 Satellite design presets

Identifier	Name	Architecture	GPU	Cells	Coating	Battery	Workload	Power W
baseline	Balanced LEO-DC	Single truss	8×H200	Si	WhitePaint	LiIon L	chat_serving	600
redwire	Redwire Serving Node	Flat payload bay	8×H200	GaAs	OSR	LiS M	frontier	700
compute_max	Compute Max	Twin-truss tower	12×B200	Perovskite	OSR	LiIon XL	training	800
eco_light	Eco Light	Cross-wing smallsat	4×H200	Perovskite	WhitePaint	LiFePO4 M	low_duty	450
thermal_guard	Thermal Guard	Windmill-wing bus	8×H200	GaAs	OSR	LiIon M	burst	600
wide_wing	Wide Wing	Blanket wing	8×B200	Perovskite	OSR	SolidState M	code_rag	650

Model 2 Constellation entity model**Table 27 Constellation entity model**

Entity name	Constellation entity
Entity class	Constellation
Composition	Several members sharing one design
Components	Walker generation component; fleet propagation component; aggregate metrics component; ground station analytics component
Interfaces	Propagate fleet propagation SunEclipse per-satellite sunlight SolarPower per-satellite generation GroundAnalytics ground station aggregate metrics
Entity parameters	Reference TLE; number of planes; satellites per plane; phasing factor; inter-satellite and ground link configuration; tracked member index
Presets	Starlink Shell 1, 72 planes × 22 satellites; OneWeb, 18 × 36; GPS Block IIR, 6 × 4; Iridium NEXT, 6 × 11; ISS single satellite; dawn-dusk Sun-synchronous, 2 × 6; user-defined design
State output	Number in orbit; number in sunlight; aggregate generated power; link counts; coverage fraction; positions of the first 256 members; full state of the tracked member

Model 3 Ground station entity model**Table 28 Ground station entity model**

Entity name	Ground station entity
Entity class	GroundStation
Composition	Single station
Components	Visibility component; downlink component; analytics component
Interfaces	Access visibility and access windows GroundAnalytics aggregate metrics
Entity parameters	Geodetic longitude, latitude and altitude, Singapore by default; elevation mask; communication band, one of UHF, S, X and Ka; solar histogram bin count
State output	Visible flag; receive rate; visible satellite count; aggregate bandwidth; cumulative elevation distribution; solar collection histogram; access windows

6.1.6 Thermal-Compute Feedback Loop

The relation between the payload components is a feedback loop that closes once per step, as shown in Figure 12. Orbit and lighting set the generation conditions, the solar array and battery give the available power, the GPU payload runs within the power budget and hands its realized draw to the radiator and structure as heat, and the structure temperature in turn caps the GPU power budget and derates the solar cell efficiency. When the radiator degrades the structure warms, the power budget shrinks, frequency, power and throughput fall together, the heat production drops, and the loop converges in a throttled state. Changing the coating or the area of one radiator panel shows up directly as a computable change in inference throughput.

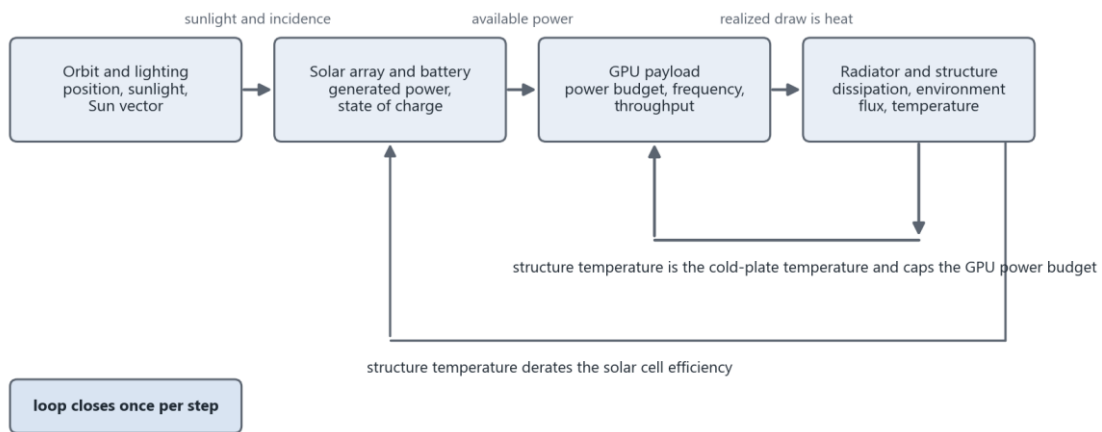


Figure 12 Thermal-compute feedback loop

6.2 Simulation Engine Subsystem

6.2.1 Engine Structure and Runtime Conventions

The simulation engine is the kernel that provides simulation advance control and simulation model management. It receives and parses the scenario parameters from the front end, calls the entity and algorithm models, and sends the results to the front end for display. The engine holds all state entities: the tracked satellite state, the ground station state, the constellation snapshot, the mission state, the configuration, the geometry, the workload profile, the attitude and the ground station target.

The engine follows three conventions. First, the engine is the single authoritative source of business state; the application layer only displays, the renderer only projects, and none of the three integrates on its own, so they never drift. Second, every state-changing method is synchronous and is called only on the event loop thread, network writes are serialized through coroutines, and commands never interleave with a step. Third, the state snapshot is

produced by model copy, so serialization for broadcast is unaffected by the next integration step.

The engine lifecycle mirrors the model standard interface. Initialization corresponds to construction and geometry restore, simulation advance to the per-step physics update, command response to the state-changing methods, and state output to the snapshot method. The offline twin of the comparison module inherits the live engine class and overrides only the fleet propagation method, replacing fleet propagation with a private single-satellite propagation. No physics formula is duplicated, so a change to any formula takes effect on every path.

6.2.2 Model Initialization Module

Model initialization runs at backend start. First, the architecture, cluster count, radiator dimensions and per-slot GPUs are restored from the geometry parameter file without increasing the version number, so that the physical areas match the model on disk. Second, the tracked satellite entity, the ground station entity and the constellation entity are assembled, and the GPU, material, battery and workload profile tables are loaded. Third, the simulation time base is set to the mission epoch and the 1 Hz stepping coroutine is started. Retired GPU identifiers from earlier catalogs are migrated to the current catalog at initialization and written back to disk.

6.2.3 Real-Time Stepping Module

The stepping module advances all models of the scenario in real time; the processing order of one physics step is shown in Figure 13.

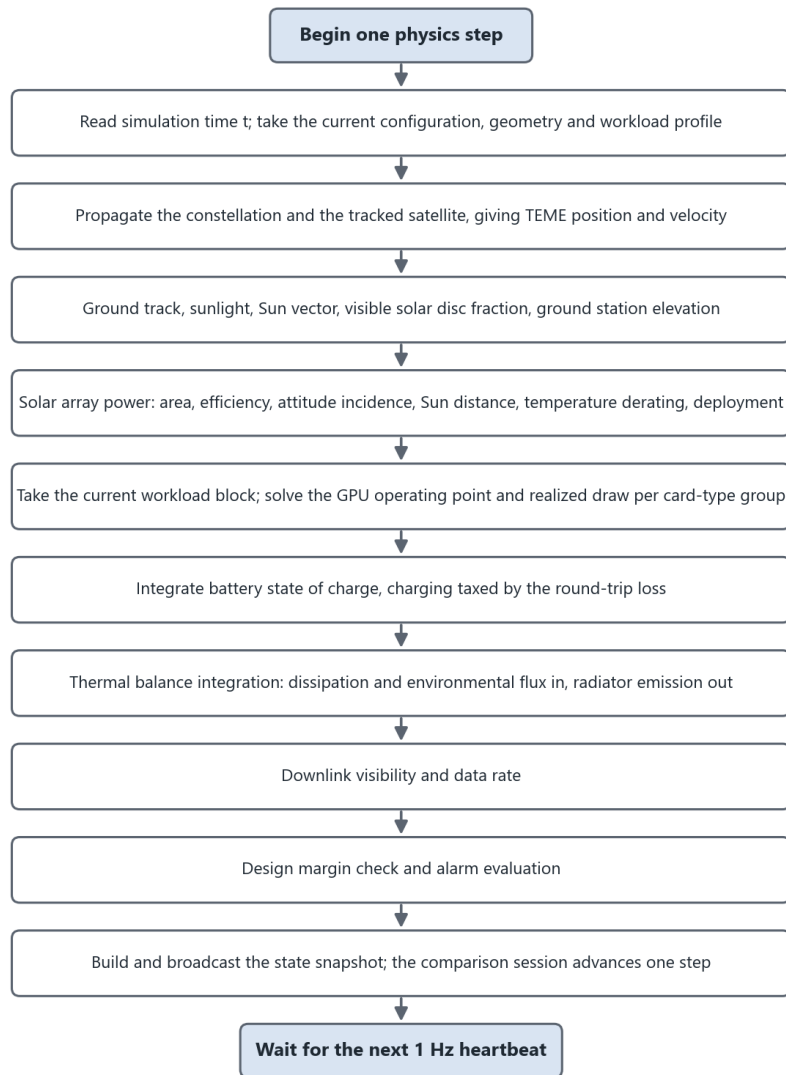


Figure 13 Real-time stepping flow

Step 1 reads the simulation time, takes the current configuration, geometry and workload profile, and derives the solar array and radiator areas from the geometry. Step 2 propagates all constellation members and obtains the position and velocity of the tracked satellite. Step 3 computes the ground track, sunlight, Sun vector, visible solar disc fraction and ground station elevation. Step 4 computes the incidence for the attitude mode and the generated power from deployment, Sun distance and array temperature. Step 5 takes the current job block, solves the GPU operating point and realized draw per card-type group, and accumulates output. Step 6 integrates the battery state of charge. Step 7 integrates the thermal balance. Step 8 determines the downlink. Step 9 computes the design margins and evaluates the alarms. Step 10 builds and broadcasts the state snapshot, and the comparison

session advances one step. The step body is protected against exceptions; a transient propagation failure skips one physics step and does not terminate the coroutine.

The time scales are aligned by ratio. The master clock, the workload schedule and the cumulative output run at 1 times simulation time; the orbit is accelerated 60 times, so one low Earth orbit takes about 90 s of wall-clock time; and the battery and thermal integration are also accelerated 60 times, so one eclipse integrates exactly one full real discharge, and entering eclipse, discharging and alarming are consistent in time.

6.2.4 Command Response Module

The command response module injects the commands of the user and of external systems into the entity models; all commands and their effects are listed in Table 29. Every command enters the single authoritative source, and the changed state returns to the three ends by broadcast or polling; the interface never predicts the result itself.

Table 29 Command response interfaces

Engine method	Network entry	Effect
play, pause, reset, set_time	WebSocket	Run control and time setting
set_config	POST /satellite_config, WebSocket set_config	GPU model and per-slot payload, cell material, coating, battery chemistry and pack size; effective at the next step
set_twin_geometry	POST /twin_geometry	Architecture, cluster count, radiator span and aspect ratio; triggers USD regeneration and version increment
apply_design	POST /designs/{id}/apply	Atomic switch of configuration, geometry, workload profile and platform power
apply_build	POST /satellite_build	Atomic commit of platform architecture, hardware configuration, per-slot payload, workload profile and attitude
set_workload_profile	POST /workload_profile	Switches the workload profile and resets the cumulative output
set_attitude_mode	POST /attitude_mode	Sun, nadir, ram and inertial pointing modes; selecting a mode zeroes the reaction wheels
set_attitude_spin	POST /attitude_spin	X, Y and Z reaction wheels; spinning up returns the pointing mode to free
set_solar_deploy	POST /solar_deploy	Deploys, retracts or toggles the roll-out solar array
set_constellation	POST /constellation/{id}	Switches the constellation preset and re-propagates
orbit design	POST /orbit_design	Synthesizes the reference TLE from orbital elements and Walker parameters, registers it as the custom preset and activates it
set_ground_target	POST /ground_target	Ground station position, elevation mask, communication band and solar bins
start_mission, stop_mission	POST /mission/start, /mission/stop	Starts and stops the mission phase machine
set_compare_session	POST /compare/start, /compare/stop	Creates and ends the comparison session

6.2.5 State Output Module

The state output module converts the engine state into the state packet. The top level of the packet contains the simulation time, the running flag, the camera preset, the design identifier, the workload profile identifier, the tracked satellite state, the ground station state, the constellation snapshot, the mission state, the ground station target state and the comparison session state. On every snapshot read the module recomputes the position-type fields of the tracked satellite, including latitude, longitude, altitude, sunlight, generated power and charging power, from the simulation time plus the wall-clock fraction, while the integrated fields keep their value from the last step, so the 5 Hz poller sees continuous motion. Sunlight, generation and charging are recomputed with the same formula on read, which avoids contradictory samples at the eclipse boundary.

6.2.6 Comparison Simulation Module

The comparison simulation module answers how the evolution differs when one design variable is changed at the current instant. The user selects one comparison dimension and 2 to 4 candidate values; the dimensions are radiator coating, cell material, GPU model, wing segment count, radiator span, battery chemistry and pack size, workload profile and whole-satellite design. At start the module synchronously freezes a live snapshot on the event loop. The 1 Hz step and the snapshot read run on the same loop and cannot interleave, so all variants are seeded from the same instant with identical orbit phase, state of charge and structure temperature. Each variant is an offline engine instance; after the candidate value is applied all deployables are forced open, and from then on the live engine calls the session to advance one step after every physics step, so pausing the simulation pauses the comparison. The current sample of each variant rides on the state packet in the broadcast, and the front end accumulates the samples into curves on top of the telemetry charts. The comparison never changes the live state and never touches the USD model.

The same mechanism serves the preview of the satellite builder wizard. On every edit of the draft the backend assembles the draft on an independent engine instance and asks it for the workload profile fit results, returning derived metrics and per-profile verdicts from the same source as the live panels, so the check seen before Run is the result obtained after Run.

6.2.7 Time System

The whole system has one authoritative clock, the simulation time of the backend physics step. Every other time is a downsampled or smoothed view of it, forming a five-level cascade in which each level smooths the steps of the level above, as shown in Figure 14 and Table 30. The renderer never advances simulation time; it only consumes it.

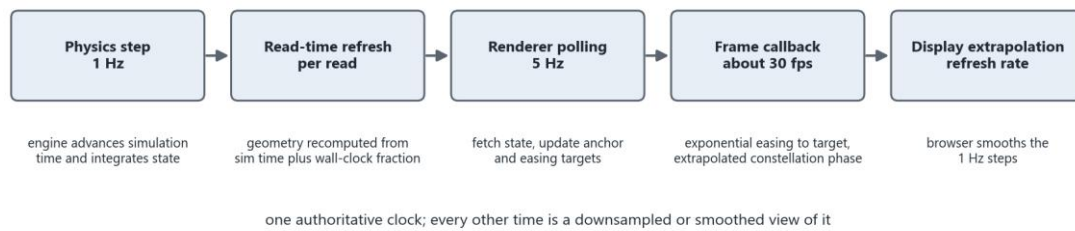


Figure 14 Time cascade

Table 30 Time cascade

Level	Executor	Rate	Mechanism
1	Backend stepping coroutine	1 Hz	The only place that advances simulation time and integrates state of charge, temperature and cumulative output
2	Backend snapshot method	Per read	Recomputes geometry and instantaneous power from simulation time plus the wall-clock fraction; integrated quantities untouched
3	Renderer polling coroutine	5 Hz	Fetches state, updates the time anchor and the easing targets
4	Renderer frame callback	About 30 fps	Close-up stage eases exponentially to the target with a 0.35 s time constant and snaps on jumps above 20° latitude or 40° longitude; overview stage extrapolates the constellation phase from the anchor
5	Front-end smoothing clock	Display refresh rate	Animation-frame extrapolation removes the 1 Hz steps

6.3 Data Exchange Subsystem

6.3.1 Unified Envelope

All messages between the browser, the backend and the renderer share one envelope containing the message type, a timestamp, the payload and an optional request identifier; requests and responses carry the same request identifier. The front end needs a single message handler for dispatch, the backend parses the envelope with one Pydantic model and handles it by type in one place, the logs are readable from the type column alone, and no new message template is needed when a command is added for the renderer. The envelope is:

```
{ "type": "<command>", "ts": 1713720000.123, "payload": { ... }, "request_id": "uuid4" }
```

An envelope that fails to parse returns a message of type error with code bad_envelope, an unknown type returns code unknown_type, and an acknowledgement always carries the request identifier. After a WebSocket disconnect the front end reconnects after 2 s.

6.3.2 WebSocket Broadcast Module

The backend serves WebSocket at the path /ws/state. When a connection is established one state snapshot is pushed first, and state_update messages follow continuously at 1 Hz. The connection manager protects the client set with an asynchronous lock and drops stale

connections automatically. The commands accepted and the messages sent are listed in Table 31.

Table 31 WebSocket messages

Direction	Type	Payload	Response or rate
Browser to backend	play, pause, reset	empty	ack
Browser to backend	set_time	sim_time_s	ack
Browser to backend	set_config	partial update of satellite configuration fields	ack and configuration echo
Browser to backend	set_parameters, set_mode	partial parameter update, run mode	ack
Browser to backend	start_task	case_id	task_update
Browser to backend	select_object	prim_path	selection_changed
Browser to backend	change_camera	preset	camera_changed
Backend to browser	state_update	state packet	continuous at 1 Hz
Backend to browser	task_update	task state	on state machine transitions
Backend to browser	selection_changed, camera_changed	pick path and details, camera preset	command echo
Backend to browser	ack, error	request identifier and result, error code and message	per request

6.3.3 REST Interface Module

The REST interface module is implemented with FastAPI. Every interface validates the request body by type, returns 400 or 422 for an invalid request and returns its result as JSON. The interfaces are grouped by function in Table 32. The renderer and external systems address the backend by the loopback address 127.0.0.1, which avoids the delay of about 0.2 s per request caused by an IPv6 fallback of host name resolution.

Table 32 REST interfaces

Group	Method and path	Function
State and catalogs	GET /health	Health check and current simulation time
	GET /state	Current state packet; polled by the renderer at 5 Hz
	GET /orbits, GET /orbits/{mode}, POST /orbit_type/{mode}	Orbit catalog and orbit type switch
	GET /constellations, GET /constellations/{id}, POST /constellation/{id}	Constellation preset list, 128-point orbit rings and preset switch
Configuration and geometry	GET, POST /satellite_config	Read and modify the hardware configuration

Group	Method and path	Function
	GET, POST /twin_geometry	Read and modify the deployable geometry; triggers USD regeneration
Design and build	GET /designs, POST /designs/{id}/apply, GET /designs/{id}/preview.png	Design library list, apply and thumbnail
	GET /satellite_assets, GET /satellite_assets/{id}/preview.png	Satellite platform catalog and thumbnail
	POST /satellite_build/preview, POST /satellite_build	Builder draft preview and atomic commit
Workload	GET /workload_profiles, POST /workload_profile	Workload profile list with fit assessment; profile switch
Orbit and ground station	GET, POST /orbit_design	Read the current constellation elements; apply an orbit design
	GET /comms_bands, POST /ground_target, GET /ground_visibility	Band catalog, ground station configuration, full-orbit pass analysis
Comparison	GET /compare/options, POST /compare/start, POST /compare/stop	Comparison dimensions and candidates; start and end a session
Runtime operations	POST /solar_deploy, POST /attitude_mode, POST /attitude_spin	Solar array deployment, pointing mode, reaction wheels
Mission and renderer	POST /mission/start, POST /mission/stop, POST /mission/scene_ready	Mission start, stop and scene-ready gate
	POST /selection	Renderer pick report; broadcasts selection_changed
WebSocket	/ws/state	State broadcast and command channel

6.3.4 Renderer Polling Interface

The Python runtime of the renderer has no WebSocket client, so the renderer obtains state by HTTP polling. The scene extension calls GET /state at 5 Hz, updates the anchor of simulation time and wall-clock on every frame received, and derives the current simulation time on every render frame from the anchor plus the wall-clock increment. The backend refreshes the kinematics of the tracked satellite on every read, so the renderer sees continuous motion. The renderer also reads GET /satellite_config and GET /twin_geometry for the material variant and the geometry version and reloads the model when the version changes, reports component picks with POST /selection, and signals the completion of mission stage loading with POST /mission/scene_ready.

6.3.5 Data Entity Contract

All cross-process data on the broadcast link are Pydantic entities defined by the backend and mirrored by TypeScript types of the same names in the front end. The top-level broadcast entity is the state packet; the world seen by all three ends is this one entity. The main entities are listed in Table 33.

Table 33 Data entity contract

Entity	Key fields	Writer	Reader	Link
StatePacket	Simulation time, running flag, camera	Engine, every	Browser, renderer	Broadcast and

Entity	Key fields	Writer	Reader	Link
	preset, design identifier, workload profile identifier and all entities below	step		polling
SatelliteState	Latitude, longitude, altitude, sunlight and Sun angle, power triplet, state of charge, temperature, alarms, deployment, attitude, osculating elements, job detail	Engine, every step	Telemetry panels, renderer drive	With StatePacket
SatelliteConfig	GPU, per-slot payload, cell material and size, coating and radiator size, battery chemistry and pack size	Configurator, builder wizard	Engine, renderer materials	WebSocket and REST
TwinGeometry	Architecture, cluster count, radiator span and aspect ratio, version	Geometry controls, design apply	Engine, generator, renderer	REST and USD regeneration pipeline
GpuJobDetail	Job and model, solver engine, realized draw, frequency, die temperature, throttle flag, group breakdown	LLM engine, every step	GPU and workload panels	With SatelliteState
FleetSnapshot	Count in orbit, count in sunlight, aggregate generation, link counts, member positions	Engine, every step	Overview page	With StatePacket
GroundTargetState	Station position, mask, band, visible count, aggregate bandwidth, elevation distribution, solar histogram	Engine, every step	Workbench tabs	With StatePacket
CompareLiveState	Comparison dimension, variant list and current sample of each variant	Comparison session, every step	Telemetry overlay	With StatePacket
DesignPreset	Configuration, geometry, workload profile, platform power	Static definition	Design library, apply pipeline	REST
SatelliteAsset	Platform architecture, slot count and grouping, factory configuration	Static definition	Builder wizard, preview and commit pipeline	REST

6.4 Situation Display and Simulation Management Subsystem

6.4.1 Overall Page Structure

The front end is implemented with React 19 and TypeScript. State is managed by one Zustand store, and all pages share one WebSocket connection and one persistent video element. The page structure is given in Table 34. The front end performs no physics and keeps no state in URL parameters.

Table 34 Page structure

Page	Route	Capabilities
Overview page	/	3D Earth and constellation, 14-parameter telemetry cards, constellation preset switch, four-tab orbit and ground station workbench, event stream
Satellite twin page	/satellite	Opens as the satellite builder wizard; after Run it is the main view: 3D twin viewport, satellite configurator, design library, comparison panel, 120 s telemetry strip

The application shell provides the brand and navigation at the top, the simulation clock, the connection status indicator and the play, pause and reset buttons. The video element is mounted once in the shell and overlaid with fixed positioning on the viewport slot of the current page, so the video element is not unmounted on page changes and the WebRTC connection is preserved. The first connection to the renderer takes about 13 s.

6.4.2 Overview Page

The overview page is the constellation-level view. The viewport shows the 3D Earth and the constellation point cloud, and the cards on the right show the live telemetry of the tracked satellite and the ground station: orbit type, latitude and longitude, altitude, lighting, solar input, payload and platform power, GPU model and utilization, temperature, state of charge, downlink bandwidth, ground station visibility and mission state, colored in three grades of normal, warning and fault when a value leaves its range. The event stream at the bottom shows recent mission transitions, errors and selection changes. After a constellation preset switch, the 3D constellation, the coverage map and the tracked satellite of the physics engine are all re-propagated at the next step.

6.4.3 Orbit and Ground Station Workbench

The right column of the overview page is a four-tab workbench. The design tab edits the semi-major axis, eccentricity, inclination, right ascension of the ascending node, argument of perigee and mean anomaly together with the Walker plane count, satellites per plane and phasing factor, shows the current constellation readings live, and on apply the backend synthesizes the reference TLE and activates it. The input ranges are altitude 200 km to 40000 km, eccentricity at most 0.5, perigee altitude at least 160 km, at most 36 planes, at most 60 satellites per plane, and a phasing factor smaller than the plane count. The design tab also configures the ground station: it marks Singapore on the Earth and selects the elevation mask, the communication band and the solar histogram bin count, with submissions debounced by 200 ms.

The coverage tab shows the status map and the curves of visible satellite count and aggregate bandwidth against time. The solar tab shows the per-satellite solar collection histogram binned by illumination intensity. The band tab shows the throughput of each communication band against the elevation mask with the current operating point marked; a higher band gives more throughput per satellite but needs a higher elevation, so the band choice trades bandwidth against visible satellite count. The full-orbit pass analysis gives access windows, coverage fraction and the countdown to the next pass.

6.4.4 Satellite Twin Page

The satellite twin page is the single-satellite view, shown in Figure 15. The viewport shows the satellite close-up streamed from the renderer; the Earth turns with the ground track, the

Sun and the key light sweep with the true zenith-to-Sun angle, and eclipse entry and exit appear in the viewport. Clicking a server blade, a solar wing, a radiator panel or a backbone part opens the corresponding information card, and the selection state stays consistent between the browser and the renderer. Before the video stream is ready the viewport shows a connecting notice, and when offline it switches to a local Three.js fallback scene in which the tracked satellite moves along the SGP4 propagated orbit ring.

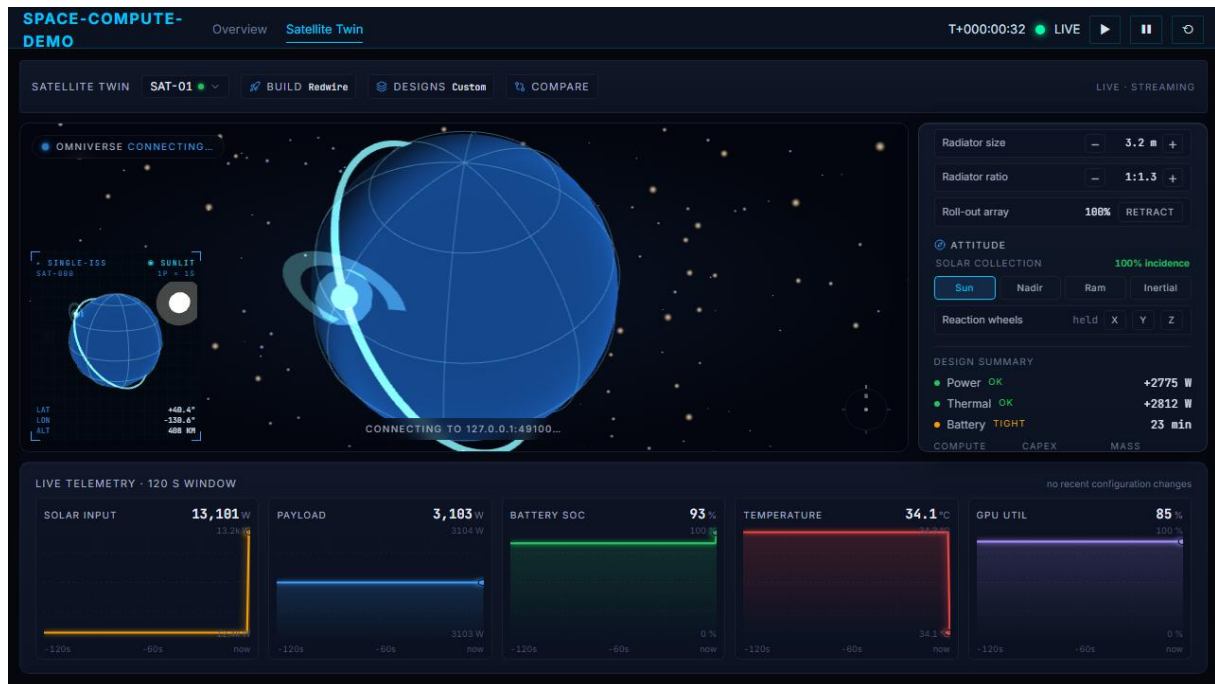


Figure 15 Satellite twin page

The configurator on the right has six sections: compute, solar array, thermal, battery, deployables and attitude. The compute section selects the GPU and hosts the workload profile selector; the workload panel shows the current job, model, utilization, effective compute, rate, electrical power, heat per card, radiated power and cumulative output. The solar array section selects the cell material and the wing segment count, the thermal section selects the coating, the radiator span and the aspect ratio, and the areas are derived live from the input. The battery section selects the chemistry and the pack size and shows the capacity in kWh. The deployables section deploys and retracts the roll-out solar array. The attitude section offers the Sun, nadir, ram and inertial pointing modes and the X, Y and Z reaction wheels, which are mutually exclusive, and the solar collection reading shows the current incidence live. The design summary shows the power, thermal and battery margins.

The telemetry strip at the bottom shows six panels in a 120 s rolling window: solar input, payload power, state of charge, temperature, GPU utilization and die temperature. Eclipse segments are shaded, variants are overlaid in different colors while a comparison runs, and the temperature panel draws the throttle onset temperature reference line of each variant.

6.4.5 Satellite Builder Wizard

Entering the satellite twin page first shows the four-step wizard. Step one selects the platform from SpaceX Compute Bus, Redwire Payload Bay, Sophia Space TILE and Ada Space Compute Node; each platform maps to one hull architecture and carries a slot count, and the factory configuration is a design preset already verified for that architecture. Step two designs the structure: cell material, wing segment count, battery chemistry and pack, coating, radiator span and aspect ratio, and the orbit pointing mode. Panel dimensions are entered in real units, wing segments 1 to 8, radiator span 0.54 m to 5.4 m, aspect ratio 1.2 to 6, clamped when out of range. Step three designs the payload by selecting a card type and then clicking slots; slots can be left empty or mixed across models, and the regenerated model renders only the blades that carry a card, with a color tag per card type. Step four selects the workload profile.

Before Run the wizard only edits a front-end draft. Every edit calls the preview interface after a 250 ms debounce, and the backend assembles the draft on an independent engine instance and returns the derived metrics and per-profile fit verdicts. On Run, the platform architecture, hardware configuration, per-slot payload, workload profile and attitude are committed atomically in one call, the USD model is regenerated and the renderer reloads. After closing, the wizard can be reopened from the build entry in the top bar.

6.4.6 Design Library and Comparison Panel

The design library window lists the six satellite designs, each with an offline software-rendered thumbnail and a derived parameter card. Clicking apply switches the hardware configuration, the deployable geometry, the workload profile and the platform power in one step, and any manual parameter change afterwards marks the current design as custom.

The comparison panel provides the dimension selector, candidate value chips, the start and stop buttons and the live value table per variant. If the current value is absent from the candidate list it is added automatically as the first candidate. After start the variant curves are overlaid as solid lines on the panels of the telemetry strip, the chart headers show the current reading of each variant, throttled values are shown in red, and after stop the live curves return.

6.4.7 Scenario Data Structures

A scenario consists of the satellite configuration, the twin geometry, the workload profile identifier, the orbit design and the ground station configuration, all expressed in JSON. The structures of the satellite configuration and the twin geometry are given in Table 35, and those of the orbit design and the ground station configuration in Table 36. An empty per-slot payload list means a uniform GPU model with a card count; a non-empty list is authoritative and the card count equals the number of fitted slots.

Table 35 Satellite configuration and twin geometry

Entity	JSON structure
SatelliteConfig	<pre>{ "gpu": "H200", "gpu_slots": ["H200", "H200", null, "B200"], "solar_material": "GaAs", "solar_size": "M", "radiator_material": "OSR", "radiator_size": "Standard", "battery_material": "LiIon", "battery_size": "L" }</pre>
TwinGeometry	<pre>{ "architecture": "truss", "solar_clusters_per_side": 5, "radiator_long": 1.9, "radiator_ratio": 2.5, "version": 12 }</pre>

Table 36 Orbit design and ground station configuration

Entity	JSON structure
OrbitDesign	<pre>{ "a_km": 6928.1, "e": 0.001, "inc_deg": 53.0, "raan_deg": 0.0, "argp_deg": 0.0, "m_deg": 0.0, "planes": 6, "sats_per_plane": 11, "phasing": 1 }</pre>
GroundTarget	<pre>{ "enabled": true, "name": "Singapore", "lat": 1.3521, "lon": 103.8198, "elevation_mask_deg": 10, "band": "X", "solar_bin": 10 }</pre>

6.5 3D Rendering Subsystem

6.5.1 Host and Extensions

The renderer uses Omniverse Kit as the host application. The kit configuration file defines the RTX renderer, the 30 frames per second update rate and the streaming layer at 1280×720. The host provides four engine services to the extensions: the frame event stream, the USD stage and context, the RTX renderer and the WebRTC streaming layer. All business logic is written in extensions that register with the host at start, listed in Table 37.

Table 37 Renderer extensions

Extension	Status	Responsibility
space.demo.scene	Active	Orchestration hub: 5 Hz backend polling, whole-stage switching, per-frame drive of Earth, Sun, lights, three-axis attitude, roll-out wing and constellation point cloud, mission orchestration, geometry hot reload
space.demo.core	Active	Opens the initial stage at start
space.demo.selection	Active	Subscribes to USD pick events and reports them to the backend
space.demo.messaging	Active	Control message channel between the browser and the renderer
space.demo.setup	Active	Window layout and startup
space.demo.camera, timeline, task_maritime	Reserved	Named cameras, clock mirror and mission overlay; their duties are currently carried by the scene extension

6.5.2 Stages and Units

The overview stage and the satellite close-up stage are switched by full replacement and are independent, so the close-up view keeps no overview resources. The two stages use different units: the overview stage uses 100 km per unit for a combined view of the Earth and the orbits, and the close-up stage uses 1 cm per unit with a root scale of 180, so 1 m of the asset corresponds to 1.8 m of the stage. The stages are listed in Table 38.

Table 38 USD stages

Stage	Content	Units
Overview stage	Earth, star field, constellation point cloud, orbit rings, ground station marker	1 unit is 100 km
Satellite close-up stage	Lights and camera layer referencing the satellite model layer; Earth sphere, cloud sphere, Sun disc and star field	1 unit is 1 cm, root scale 180
Mission stage	Maritime mission overlay	Same as the close-up stage

Two rendering conventions apply to all extensions. First, per-frame animation is written only into the USD session layer, so the model file on disk stays clean and geometry hot reload never conflicts with per-frame writes. Second, heavy assets such as solar panels, radiator panels and servers use class prototypes with instanceable references, so the renderer composes them once.

6.5.3 Per-Frame Drive and Time Anchoring

The scene extension registers two clock sources. The polling coroutine fetches state at 5 Hz and updates the anchor of simulation time and wall-clock, and the frame callback at about 30 frames per second derives the current simulation time from the anchor plus the wall-clock increment. The close-up stage eases exponentially toward the target with a 0.35 s time constant, longitude follows the shortest arc, and jumps above 20° of latitude or 40° of

longitude snap directly to avoid camera swings. The overview stage recomputes the constellation phase by extrapolation from the anchor, and the Earth rotation follows the wall-clock. A flip of the running flag or a simulation time jump above 1.5 s forces a re-anchor.

In the close-up stage the Earth sphere rotation is driven by the ground track, the Sun disc and key light sweep with the zenith-to-Sun angle, the satellite body eases to the target attitude of the pointing mode, the attitude angle is integrated per frame when the reaction wheels spin, and the roll-out wing stretches from its root anchor with the deployment fraction. When the geometry version changes the model layer is reloaded with a forced reload, and the ordering guarantees that the renderer never reads a half-written file.

6.5.4 Streaming and the Three Viewport States

The streaming layer pushes 720p video and input events to the browser over WebRTC. The browser viewport has three states. When the video stream is ready it shows the renderer image; while connecting it shows a notice; when offline it shows the local Three.js fallback scene, which also follows the SGP4 propagated orbit. When the renderer window is minimized the operating system suspends its render loop and the encoder has no frames to encode, so the renderer window stays visible during local debugging and the renderer is started in windowless mode for unattended deployment.

6.6 3D Asset Subsystem

6.6.1 Parametric Model Generation

The satellite model is generated by the parametric generator from the geometry parameter file. The generator uses the backbone asset as the spine. The backbone contains a vertical truss, thruster modules at both ends, a tank and reaction wheel assembly in the middle, and four quadrant racks with three open slots each, giving 12 server blade slots. The solar wings deploy along $\pm Y$ as several 2×2 panel clusters, the cell faces point at the Sun, and clusters can only be added outward. The radiator panels deploy along $\pm Z$, perpendicular to the wings, with adjustable size and aspect ratio. The six hull architectures supported by the generator are listed in Table 39.

Table 39 Hull architectures

Architecture	Identifier	Description	Solar array area	Radiators
Single truss	truss	Vertical truss with 12 blade slots	From cluster count	Two panels, two faces
Twin-truss tower	twin_truss	Two backbone segments stacked into a 24-slot tower	From cluster count	Two panels, two faces
Blanket wing	blanket	Single-row long blanket wings	From cluster count	Two panels, two faces
Cross-wing smallsat	lumid	Smallsat hull with integrated cross wings	Fixed 9.5 m ²	Two panels, two faces

Architecture	Identifier	Description	Solar array area	Radiators
Windmill-wing comms bus	dish	Parabolic dish with windmill wings	Fixed 18.0 m ²	Two panels, two faces
Flat payload bay	redwire	Bay carries 7 GPU modules, flat roll-out wings coplanar with the deck	From cluster count	One panel, two faces

Areas are derived from the geometry parameters and agree with the generator constants. One cluster is about 3.76 m², and the solar array area is the clusters per side times 2 times the cluster area. The long edge of a radiator panel is the span parameter times 1.8, the short edge is the span divided by the aspect ratio times 1.8, and the area is the sum over two panels and two faces. Blades that carry a GPU get a color tag on the leading edge, and the tag colors match the slot colors of the front end.

6.6.2 Geometry Parameters and the Hot Reload Pipeline

A geometry command is processed in this order: the backend writes the geometry parameter file under a lock, calls the generator in a subprocess to regenerate the model layer, increments the version number after the file is written, and the renderer detects the version change in its next polling period and force-reloads the model layer. The version number increases only after a successful write, so the renderer never reloads a partial file. Restoring the geometry from disk at start does not increment the version.

6.6.3 Offline Software Renderer and Asset Rules

The offline software renderer is implemented with pxx, numpy and PIL without any graphics API. It rasterizes any stage with the painter's algorithm and flat shading and produces the thumbnails of the design library and the platform catalog as well as geometry check renders. Render time grows linearly with triangle count, so thumbnails use box stand-ins. Thumbnails are cached by geometry fingerprint and pipeline version and are committed to the repository.

Raw assets are kept in internal, public and vendor groups and enter the production directory after conversion, structural normalization, asset validation and scene optimization. Only CC0, CC-BY, public domain, MIT and NVIDIA SimReady licenses are accepted, non-commercial-only assets are excluded, and every source and license is recorded in the asset manifest. The large backbone, solar panel, radiator and hull assets are not committed to the repository and are kept locally.

6.7 Verification Design

6.7.1 Validators

Six validators and two benchmark pipelines guard physical correctness, listed in Table 40. Every validator prints a pass and fail summary and exits with a non-zero code on failure. The

validators that depend on the backend connect to port 8001 by default and accept a port argument to target an isolated instance.

Table 40 Validators

Script	Coverage	Checks	Dependency
validate_physics.py	Full-orbit power and thermal balance of the six design presets	6 presets $\times$ 9	Running backend
validate_attitude_solar.py	Full-orbit lighting sweep per attitude mode, eclipse gating, derived battery capacity	16	Running backend
validate_compare.py	Consistency of the offline comparison twin with the live engine, lockstep and conservation laws	14	Running backend
validate_fleet_solar.py	Fleet aggregate generation consistent with single-satellite generation	Regression	Running backend
validate_llm_perf.py	LLM operating point analytic solver, exact anchor reproduction	52	None
validate_llm_engine.py	Thermal and throughput closed-loop scenarios	32	None
validate_elements.py	Conversion between orbital elements and state vectors	27	None
tools/stk_benchmark	Orbit, power and thermal against STK 11.6 truth	51	STK 11.6 with SEET
tools/comsol_benchmark	Whole-satellite lumped thermal model against COMSOL 6.3 finite elements	Two cases	COMSOL 6.3 with MPh

6.7.2 Benchmark Against STK 11.6

The benchmark pipeline takes the local STK 11.6 headless engine with the Space Environment and Effects Tool as truth. The single authoritative source of scenarios and parameters is the case file; the TLEs are shared with the engine catalog, the time axis is 24 h from the engine mission epoch at a 60 s step, and the cases are an ISS-class low Earth orbit, a 705 km Sun-synchronous orbit and a geostationary orbit. The criteria follow the 17 test cases of Document 2. The STK truth is exported once, and after a change to the engine physics only the engine export and the scoring steps are rerun. The verdicts are summarized in Table 41; all 51 criteria over the three cases pass.

Table 41 STK benchmark verdict summary

ID	Metric	Threshold	Worst measured over three cases	Verdict
B1	TEME position difference	≤ 0.001 km	2.9×10^{-10} km	Pass
B1	TEME velocity difference	$\leq 10^{-6}$ km/s	3.3×10^{-13} km/s	Pass
B2	Ground track latitude and longitude difference	$\leq 0.02^\circ$	$4.5 \times 10^{-11}^\circ$, 1.1×10^{-70}	Pass
B2	Altitude difference	≤ 1 km	5.1×10^{-9} km	Pass
B3	Sun direction and β angle difference	$\leq 0.5^\circ$	0.017° , 0.003°	Pass
B4	Sunlit fraction difference	≤ 1 percentage point	0.077 percentage point	Pass

ID	Metric	Threshold	Worst measured over three cases	Verdict
B4	Eclipse event timing difference	≤ 15 s, none missed or spurious	4.8 s, 0 missed, 0 spurious	Pass
B5	Access window edges at 0°, 5°, 10°, 20° masks	≤ 1 s, none missed or spurious	1.0 s, 0 missed, 0 spurious	Pass
B6	Illumination factor level RMS	≤ 0.02	0.009	Pass
B7	24 h energy, Sun-pointing	$\leq 2\%$	0.39%	Pass
B7	24 h energy, nadir-pointing	$\leq 2\%$	0.023%	Pass
B8	Minimum state of charge difference	≤ 5 percentage points	0.0 percentage point	Pass
B9	Sunlit segment mean temperature difference	≤ 10 K	0.34 K	Pass
B9	Eclipse segment mean temperature difference	≤ 10 K	0.31 K	Pass

SGP4 position difference between the platform and STK 11.6, 24 h, 60 s step

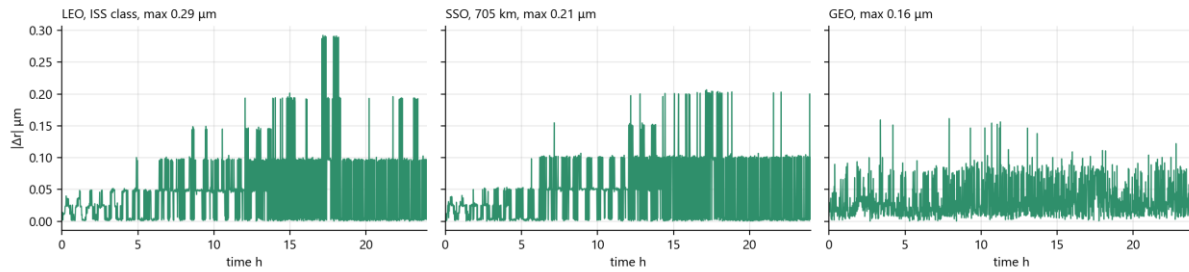


Figure 16 24 h SGP4 position difference for the three cases

Three known boundaries are recorded in the benchmark. Eclipse uses a spherical Earth, so high-latitude eclipse edges of the Sun-synchronous orbit differ from the ellipsoidal shadow by about 4 s to 5 s. Elevation carries no atmospheric refraction correction, which matches STK with refraction disabled. The STK vehicle temperature data is a steady-state equilibrium model, so the benchmark compares the instantaneous equilibrium temperature of the engine formula under the same flux terms.

6.7.3 Whole-Satellite Thermal Comparison Against COMSOL 6.3

To assess the range of validity of the single-node lumped thermal model, a whole-satellite finite element model was built in COMSOL 6.3 with the heat transfer in solids, surface-to-surface radiation and orbital thermal loads interfaces. The geometry comes from the asset bounding boxes and the generator constants, the orbit positions and the Sun vector come from the sequences exported by the engine, and the heat sources are 95% of the per-card draw of the engine. The two cases are 12 V100 cards and 12 A100 cards with white-paint radiators and the inference workload profile. The comparison over orbits 2 to 3 is given in Table 42, and the temperature curves of the V100 case in Figure 17.

Table 42 Whole-satellite thermal model against COMSOL finite elements, orbits 2 to 3

Probe	V100 max deviation °C	V100 mean deviation °C	A100 max deviation °C	A100 mean deviation °C
Radiator area-weighted mean	12.0	−9.1	16.1	−12.1
Radiator maximum	7.6	−5.1	10.5	−7.0
GPU baseplate mean	11.0	+9.7	13.8	+12.1
Bus volume mean	17.5	+15.5	20.4	+17.3
Radiator in-plane gradient, mean and max	4.0 and 4.6		5.2 and 5.8	
Frozen steady-state energy balance residual	3.63%		3.87%	

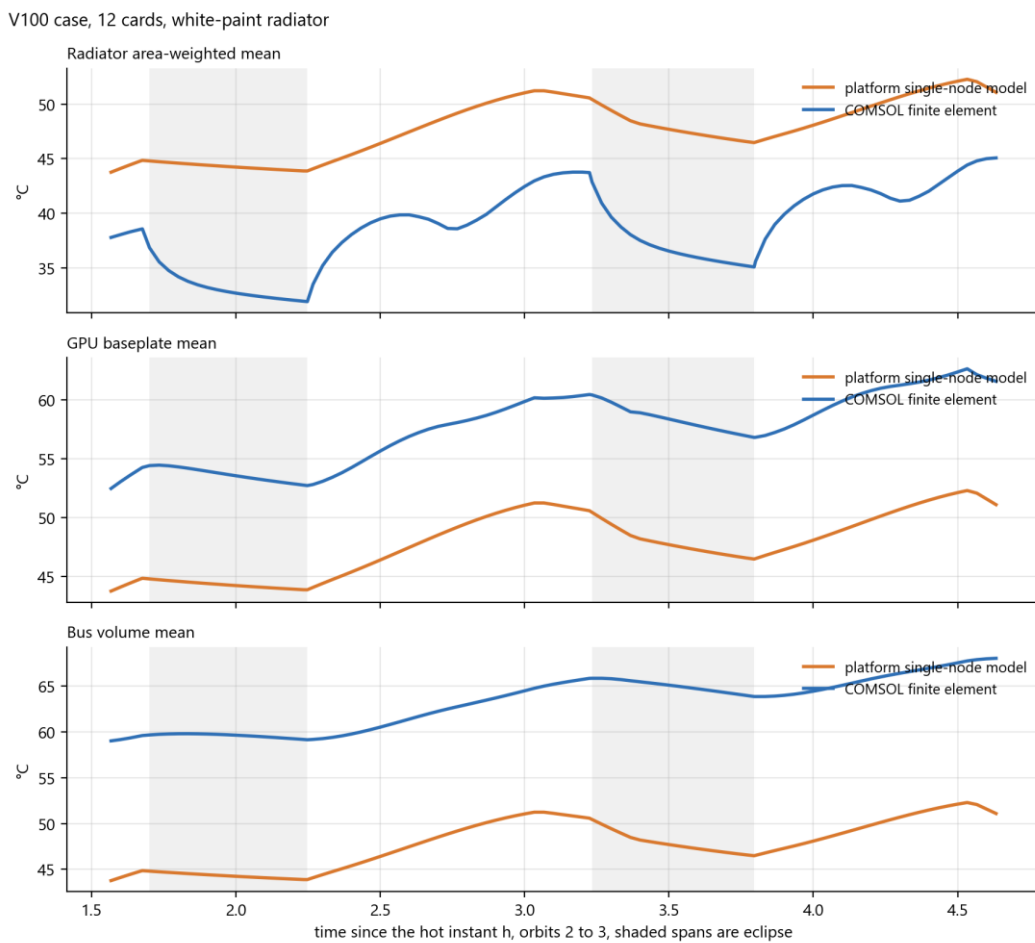


Figure 17 Temperature comparison of the single-node model and COMSOL finite elements, V100 case

The comparison shows that the single-node temperature lies between the finite element radiator and bus, 9 °C to 12 °C above the radiator surface and 10 °C to 17 °C below the electronics. The difference comes from the missing conduction path from the bus to the radiator in the single-node model; in the finite element model the heat pipes, the interface material and the bus skin produce a drop of about 25 °C to 30 °C at 2.4 kW to 3.1 kW. The thin radiator of the finite element model swings about 6 °C with each eclipse, while the 160

kJ/K single node follows only the 6 h workload cycle. The single-node model is suitable for design screening at the architecture stage; board-level thermal design needs additional baseplate and radiator nodes with conduction resistances between them, which is the planned improvement of the thermal model.

6.7.4 Interface End-to-End Tests

The front end runs end-to-end cases with Playwright covering navigation and smoke tests, orbit motion, attitude control, battery and solar array, comparison sessions, the design library, the LLM inference panel, the orbit designer, the satellite builder wizard, solar array deployment, the workload panel and screenshots, in 13 case files. The satellite twin page is covered by the wizard on entry, so every twin page case dismisses the wizard first. The cases related to streaming require the renderer to be running.

7 Requirement Compliance Status

Table 43 Requirement compliance status

No.	Indicator	Subsystem	Compliance	Status
1	Constellation ≥ 1500 satellites, 24 h position difference ≤ 1 m, ground track difference $\leq 0.02^\circ$	Simulation model and simulation engine subsystems	The Starlink Shell 1 preset has 72 planes $\times$ 22 satellites, 1584 in total, and every physics step propagates the fleet within 1 s. Over three cases the 24 h position difference is at most 2.9×10^{-10} km and the ground track longitude difference at most $1.1 \times 10^{-7}^\circ$. Met.	Complete
2	Sun direction $\leq 0.5^\circ$, eclipse timing ≤ 15 s, sunlit fraction ≤ 1 percentage point	Simulation model subsystem	Sun direction angle at most 0.017° , β angle at most 0.003° , eclipse event timing at most 4.8 s, sunlit fraction difference at most 0.077 percentage point. Met.	Complete
3	Access window edges ≤ 1 s, no missed or spurious windows	Simulation model subsystem	Window edges differ by at most 1.0 s at the four masks with 0 missed and 0 spurious windows. Met.	Complete
4	24 h energy $\leq 2\%$, minimum state of charge ≤ 5 percentage points	Simulation model subsystem	Sun-pointing energy difference at most 0.39%, nadir-pointing at most 0.023%, minimum state of charge difference 0.0 percentage point. Met.	Complete
5	Segment mean temperature ≤ 10 K	Simulation model subsystem	Sunlit segment mean difference at most 0.34 K, eclipse segment at most 0.31 K. Met.	Complete
6	1 Hz stepping, 1 Hz broadcast, 1280×720 at 30 frames per second	Simulation engine, data exchange and 3D rendering subsystems	The engine steps in a 1 Hz coroutine, WebSocket broadcasts at 1 Hz, the renderer polls at 5 Hz, and streaming is configured at 1280×720 and 30 frames per second. Met.	Complete
7	4 platforms, 4 GPUs, 4 coatings, 3 cell materials, 4 chemistries, 6 architectures, 2 to 4 variants	Situation display and simulation management and 3D asset subsystems	The builder wizard offers 4 platforms; the catalogs offer V100, A100, H200 and B200, 4 coatings, 3 cell materials and 4 chemistries; the generator supports 6 architectures; the comparison session supports 2 to 4 variants in lockstep. Met.	Complete
8	≥ 50 theory checks, ≥ 30 closed-loop checks	Simulation model subsystem	52 checks of the LLM operating point solver and 32 thermal-throughput closed-loop checks, all passing. Met.	Complete
9	REST and WebSocket interfaces	Data exchange subsystem	36 REST interfaces and 1 WebSocket interface with a unified envelope; external systems can read state and inject commands. Met.	Complete

